

SQX: bloques, plantillas y proyectos

AlgoProject — manual de uso. Actualizado el 2026-10-01 con `tools/manual.py`.

1. 40. sqx-lab — instalar y refrescar el kit de autoría sin `/plugin``
2. 36. Taxonomía y paletas de bloques — qué bloque pega con qué tipo de estrategia
3. 22. Plantillas — de una idea a una plantilla que construye
4. 28. Builder — de una plantilla a un proyecto que arranca, en un comando
5. 47. El proyecto del workflow — todas las pruebas de una estrategia en un solo custom project
6. 6. Quitar estrategias descartadas — aplicar un veredicto dentro de SQX
7. 27. Curar una databank — escribir el veredicto con un filtro o con un nombre
8. 17. Pipeline — mete cien estrategias, vuelve al cabo de unos días y lee los veredictos
9. 80. Autopiloto — el workflow de un proyecto de punta a punta, sin pararse
10. 55. Retirar proyectos — quitar de SQX los custom projects que ya respondieron
11. 62. Metadatos de una estrategia — qué hace y con qué costes se probó, leído del fichero

40. sqx-lab — instalar y refrescar el kit de autoría sin

`/plugin`

Qué pregunta responde

«¿Por qué las skills que escriben bloques, grupos, plantillas y proyectos de SQX no se instalan con `/plugin` como dice su web, y cómo me aseguro de que están bien?»

`sqx-lab` es un plugin de Claude Code del autor de StrategyQuant. Su instalación oficial son dos comandos:

```
/plugin marketplace add /home/sergioguslw/Desktop/AlgoProject/tools/sqx-lab
/plugin install sqx-lab@sqx-lab
```

y **en este entorno no existen**. Medido el 2026-09-24: la extensión de VSCode responde `/plugin isn't available in this environment`. Así que el kit se instala a mano, y este script es esa mano.

Lo único que pierdes respecto a una instalación de plugin de verdad es la actualización automática. Las cuatro skills y los dos comandos se comportan igual una vez enchufados.

Cuándo lo usas, y cuándo no

- **Sí**, la primera vez en una máquina nueva.
- **Sí**, después de bajar una versión nueva del zip y descomprimirla sobre `tools/sqx-lab/`.
- **Sí**, cada vez que cambien los bloques o los grupos del install que construye — los catálogos se quedan rancios y **un catálogo rancio no falla: inventa átomos** que no están en el install, y la plantilla sale silenciosamente mal.
- **No** para arreglar un fallo de una plantilla concreta. Para eso, `/sqx-doctor` primero.

Antes de empezar

Sólo que exista el install cuyos catálogos quieres construir. El script lo comprueba y se niega si no está. No hace falta parar SQX: sólo lee `customBlocks.xml`, `blockGroups.xml`, `user/projects` y `user/settings/StrategyTemplates`.

Cómo se ejecuta

```
bin/sqx-lab-install.sh
bin/sqx-lab-install.sh --role custodian      # construir los catálogos de otro install
```

flag	obligatorio	qué hace
<code>--role</code>	no	de qué install salen los catálogos. Por defecto conductor , que es el carril de autoría

Es **idempotente**: se puede correr todas las veces que quieras. Tarda unos segundos, salvo los catálogos, que son medio minuto.

Qué produce

ruta	qué es
<code>~/.claude/skills/sqx-*</code>	cuatro symlinks a las skills del plugin. Enlaces, no copias: así una versión nueva se ve sin volver a copiar nada
<code>~/.claude/commands/sqx-setup.md</code> , <code>sqx-doctor.md</code>	copias , y aquí está el detalle que importa
<code>tools/sqx-lab/.../catalog.json</code> y <code>catalog.md</code>	los catálogos, uno por skill, junto a su propia skill
<code>~/.sqx-lab/sqx-install.txt</code>	el install que las cuatro comparten. Lo escribe el bootstrap

⚠ **Por qué los comandos son copias y no enlaces.** Los del fabricante llaman a `"${CLAUDE_PLUGIN_ROOT}/doctor.py"` , y esa variable sólo la define una instalación de plugin de verdad. El script las copia sustituyendo la variable por la ruta real. **Ésa es la razón por la que una actualización obliga a volver a correr esto:** los enlaces se actualizan solos, las copias no.

⚠ **Se enchufan tres, no cinco.** `sqx-strategy-project` está **retirada** (dueño, 2026-09-25): clona cualquier proyecto de la instalación y despliega en ella, contra la regla dura 10, y seis de sus pruebas fallan aquí. Los proyectos los crea `sqx.projects.builder` desde el donante congelado. `sqx-spp` es local, no viene del zip, y duplica el `/spp` de este proyecto. El script quita el enlace de las dos si existe (`retired`).

⚠ **Las tres llevan encima las reglas de este proyecto.** El script inserta `tools/sqx-lab/overlays/<skill>.md` al principio de cada `SKILL.md` del fabricante, entre marcas (`overlay`), y a `sqx-strategy-template` le cambia la descripción para que tus defaults manden: tu condición fija más un aleatorio libre. Las capas se editan en `overlays/` , nunca dentro del fichero del fabricante. Hay además dos parches al código del fabricante, listados en `tools/sqx-lab/LOCAL_PATCHES.md` ; si una versión nueva se los lleva, el script lo avisa con `⚠ falta el parche local` . Descomprimir una versión nueva encima de `tools/sqx-lab/` se lleva también `sqx-spp` por delante.

Cómo se lee el resultado

Salida real del 2026-09-25:

```

sqx-lab 1.2.0 -> conductor (/home/sergioguslw/Desktop/SQX_w1)
  skill    sqx-custom-block
  skill    sqx-random-group
  skill    sqx-strategy-template
  overlay  sqx-custom-block
  overlay  sqx-random-group
  overlay  sqx-strategy-template
  command  /sqx-setup
  command  /sqx-doctor
catálogos:
  note: 74 talib_* atoms flagged UNUSABLE in single-symbol builds
  .../sqx-random-group/catalog.md
wrote .../sqx-strategy-template/engine/catalog.json
-> .../sqx-strategy-project/engine/catalog.json (8 projects, 8 templates)

=====
All green – the full block -> group -> template -> project chain is usable.

```

La última línea es la que se mira. Cualquier cosa que no sea **All green** la explica el propio doctor con la orden que lo arregla. Lo que decía antes de correr esto, el 2026-09-24:

```

[ WARN ] sqx-custom-block – built 20d ago from a DIFFERENT install:
/home/sergioguslw/Desktop/SQX

```

Veinte días rancio y construido contra el **maestro** en vez del conductor. Eso es el fallo que hay que reconocer: las skills diseñaban contra bloques de otro install.

Un ejemplo completo — actualizar a una versión nueva

```

cd /tmp && curl -sSL0 https://strategyquant.com/wp-content/uploads/2026/08/sqx-lab-1.2.0.zip
unzip -q sqx-lab-1.2.0.zip -d nuevo

# mira qué cambia antes de sobrescribir; los catalog.json y __pycache__ salen siempre
diff -rq nuevo/sqx-lab ~/Desktop/AlgoProject/tools/sqx-lab | grep -v catalog -e __pycache__

# guarda la skill local, que el copiado se la lleva
cp -r ~/Desktop/AlgoProject/tools/sqx-lab/plugins/sqx-lab/skills/sqx-spp /tmp/
cp -r nuevo/sqx-lab/. ~/Desktop/AlgoProject/tools/sqx-lab/
cp -r /tmp/sqx-spp ~/Desktop/AlgoProject/tools/sqx-lab/plugins/sqx-lab/skills/

cd ~/Desktop/AlgoProject && bin/sqx-lab-install.sh

```

Qué NO te dice

- **No dice que una plantilla vaya a construir.** Sólo que las skills ven el vocabulario correcto. Lo que construye o no lo dice el motor: `sqx/inspect/template_check.py` después del build.
- **No compara los tres installs entre sí.** Un bloque puede estar en el conductor y no en el custodio, y entonces el build ignora la plantilla sin un solo error. Eso es `python3 -m sqx.inspect.vocabulary --diff custodian`.
- **No instala nada en SQX.** Los bloques que una skill escribe se instalan con `python3 -m sqx.blocks.install`, con el install parado.

Si algo falla

- **no está el plugin en ...** — `tools/sqx-lab/` no existe o está a medio descomprimir. Baja el zip otra vez.
- **el install <rol> no existe** — el rol no está en `config/machine.yaml`.
- **Un [WARN] STALE catalog justo después de correrlo** — el install cambió entre el bootstrap y el doctor, o construiste los catálogos contra un rol distinto del que mira el doctor. Ese rol sale de `~/ .sqx-lab/sqx-install.txt`.

36. Taxonomía y paletas de bloques — qué bloque pega con qué tipo de estrategia

Qué pregunta responde

SQX conoce 1.020 bloques. Con tantos, el builder puede combinar un filtro de reversión a la media con un disparador de ruptura y sacar algo que parece rentable y no significa nada. Este módulo prepara el fichero donde se decide, bloque a bloque, **con qué familia de estrategia pega cada uno**.

No etiqueta nada: monta la tabla vacía, con todo lo que se puede saber leyendo la instalación, para que la etiqueta la ponga alguien que sepa —tú o un agente— y no se pierda en un chat.

Cuándo lo usas, y cuándo no

Se corre **cuando cambia el vocabulario de la instalación**: después de autorar bloques propios con `/sqx-strategy-template`, después de importar un paquete de bloques, o cuando llega una instalación nueva. Fuera de eso no hay motivo: el fichero ya está y las etiquetas no las pone este comando.

No sirve para saber si un bloque es bueno, ni si funciona en tu mercado. Solo dice qué bloques existen, qué papel puede jugar cada uno y qué grupos aleatorios lo agrupan.

⚠ **Y no vale para estrechar un hueco atado a un grupo**. Las etiquetas alimentan una paleta, y una paleta solo alcanza los huecos **libres** de una plantilla. Un `RandomCondition` atado a un grupo sortea ese grupo y se salta los interruptores — medido el 2026-09-24, `knowhow/authoring/builder-block-switches.md`. Para esos, la palanca es el grupo.

Antes de empezar

Nada. Lee tres ficheros XML de la instalación y una tarea Build del donante congelado. **No arranca SQX, no escribe en ninguna instalación y se puede correr con la GUI del maestro abierta**.

Lo único que tiene que existir es el donante, que ya está:

```
~/Desktop/AlgoData/projectsBackup/XAUUSD_base_2026-09-21/project.cfx .
```

Cómo se ejecuta

```
python3 -m sqx.blocks.taxonomy          # refresca desde el maestro
python3 -m sqx.blocks.taxonomy --role custodian  # leer otra instalación
```

flag	obligatorio	qué hace
<code>--role</code>	no	<code>master</code> , <code>conductor</code> o <code>custodian</code> . Por defecto el maestro , que es el que lleva el vocabulario de referencia
<code>--donor</code>	no	el <code>project.cfx</code> cuya tarea Build lista los papeles. Por defecto el donante congelado de XAUUSD. Solo lectura
<code>--out</code>	no	dónde escribir. Por defecto <code>sqx/blocks/taxonomy.yaml</code>

Tarda menos de dos segundos.

Qué produce

Un único fichero, **en el repositorio** porque es configuración a mano y tiene que estar versionada:

```
sqx/blocks/taxonomy.yaml (~148 KB, 767 bloques en 83 categorías)
```

Lo sobrescribe entero en cada corrida, pero conservando lo único que no es derivado: los `archetypes` que ya tuvieran valor. Todo lo demás —los papeles, los grupos, la forma escrita— se vuelve a leer de la instalación y se pisa.

Cómo se lee el resultado

La salida real de hoy:

```
/home/sergioguslw/Desktop/AlgoProject/sqx/blocks/taxonomy.yaml: 767 bloques en 83 categorías,
leídos de SQX
  papeles   señal 588 · indicador 171 · nivel 85
  etiquetas 0 puestas, 767 por poner
  nuevos (767): ADX, ADXChangesDown, ADXChangesUp, ADXCrossDown, ADXCrossUp, ADXFalling,
  ADXHigher, ADXLower ...
```

Tres cosas que leer ahí:

- **767 de 1.020.** Los otros 253 existen y el builder no los ve: las acciones, las funciones matemáticas, los `158 talib_*`. Etiquetarlos sería etiquetar algo que ninguna paleta puede tocar.
- **Los tres papeles.** Un mismo bloque puede jugar dos: el `ATR` es indicador (un valor que comparar) y nivel (el precio de una orden stop). Son dos interruptores distintos sobre él.
- **etiquetas 0 puestas, 767 por poner.** Ese es el trabajo que falta. Mientras esté en 0, ninguna paleta se puede construir.

Una fila del fichero:

```
BookTriggers_user:
  CBlock_BBBreakoutUp:
    roles: [signal]
    origin: own
    form: BBBreakoutUp
    groups: [BookTriggers]
    archetypes: {}
```

`archetypes` es lo que hay que rellenar: **un peso por familia**, `breakout`, `mean_reversion` y `trend`. `0` excluye el bloque de esa familia, `1` es neutro, `2` y `3` lo prefieren. Vacío = sin etiquetar, y un bloque sin etiquetar no entra en ninguna paleta.

`groups` es el aviso: ese bloque está en `BookTriggers`, así que una plantilla que ate un hueco a `BookTriggers` lo puede sacar **aunque la paleta lo apague**.

Un ejemplo completo

Etiquetar los bloques de ruptura propios y comprobar que sobreviven a un refresco.

```
$ python3 -m sqx.blocks.taxonomy
sqx/blocks/taxonomy.yaml: 767 bloques en 83 categorías, leídos de SQX
papeles   señal 588 · indicador 171 · nivel 85
etiquetas 0 puestas, 767 por poner
```

Se edita el fichero a mano y se pone una etiqueta:

```
CBlock_BBBreakoutUp:
  roles: [signal]
  origin: own
  form: BBBreakoutUp
  groups: [BookTriggers]
  archetypes: {breakout: 3, mean_reversion: 0, trend: 1}
```

Se vuelve a correr, por ejemplo después de autorar un bloque nuevo:

```
$ python3 -m sqx.blocks.taxonomy
sqx/blocks/taxonomy.yaml: 768 bloques en 83 categorías, leídos de SQX
papeles   señal 589 · indicador 171 · nivel 85
etiquetas 1 puestas, 767 por poner
nuevos (1): CBlock_DonchianBreakUp
```

La etiqueta sigue ahí y el bloque nuevo aparece vacío, esperando la suya.

Qué NO te dice

- **No dice si la etiqueta es correcta.** Es una opinión escrita en un fichero. La evidencia de que un bloque pega con una familia sale de las corridas (`runs.csv`), no de aquí.
- **No construye ninguna paleta ni toca ningún proyecto.** Escribe un YAML y nada más.
- **No alcanza los huecos atados a un grupo.** Repetido porque es el error que va a costar caro: una paleta perfecta no cambia nada en una plantilla cuyos huecos apuntan a grupos.
- **No es el catálogo completo del vocabulario.** Para eso, `sqx/inspect/vocabulary.py` y la página `01-empezar.pdf` > `21-vocabulario`.

La librería de paletas

La taxonomía dice **qué es** cada bloque. Una **paleta** dice **qué entra en una construcción concreta**, y de esas hay muchas: tres por defecto que vienen con el repositorio, una por familia, y todas las que clones o crees encima.

```
sqx/blocks/palettes/ruptura_base.yaml      ★ por defecto, familia Ruptura
sqx/blocks/palettes/reversion_base.yaml    ★ por defecto, familia Reversión a la media
sqx/blocks/palettes/tendencia_base.yaml    ★ por defecto, familia Tendencia
sqx/blocks/palettes/<lo_que_tú_guardes>.yaml
```

Una paleta son cuatro decisiones y nada más:

campo	qué decide
family	de qué columna de la taxonomía hereda los pesos: <code>breakout</code> , <code>mean_reversion</code> o <code>trend</code>
unlabelled	qué pasa con un bloque sin etiquetar — <code>neutral</code> entra a peso 1, <code>off</code> lo deja fuera
overrides	los pesos puestos a mano, que ganan sobre la taxonomía. <code>0</code> apaga el bloque
label · note · origin · based_on	cómo se llama, para qué es, y de dónde salió

unlabelled: off más una lista de **overrides** hace que la paleta SEA esa lista: no entra nada más. Es la forma de tener una paleta curada —«para ruptura yo usaría estos treinta bloques»— **sin esperar a que la taxonomía esté etiquetada**. Con `neutral` la paleta hereda lo que diga la taxonomía y solo corrige lo que tú le digas.

El comando

```
python3 -m sqx.blocks.palette                # la librería entera
python3 -m sqx.blocks.palette ruptura_base    # una paleta: qué deja pasar
python3 -m sqx.blocks.palette ruptura_base --json # lo mismo, para un agente
```

flag	obligatorio	qué hace
NAME	no	el slug de una paleta. Sin él, lista la librería
--json	no	emite la paleta y su resolución bloque a bloque, para que un agente la consuma

Salida real de hoy, con las tres por defecto y ninguna curada todavía:

```
ruptura_base      Ruptura                default  condiciones  64  ▲ FUERA DE 90-170  Ruptura
- base
reversion_base    Reversión a la media  default  condiciones  83  ▲ FUERA DE 90-170
Reversión a la media - base
tendencia_base    Tendencia            default  condiciones  98  ok
Tendencia - base
```

El número que importa es **condiciones**, y tiene banda: el dueño pide **entre 90 y 170 por build** (2026-09-24). Por debajo el builder no tiene de dónde combinar; por encima vuelve el sobreajuste que esto viene a evitar. El comando marca `▲ FUERA DE 90-170` cuando una paleta se sale, y la ventana pinta el recuento en rojo. No es una orientación: es el criterio de aceptación.

Clonar, editar, guardar

Se hace en la ventana (`02-la-ventana.pdf` > `35-app-plantillas` , zona **Paletas**): abres una, pulsas **Clonar**, le das nombre, y la copia es tuya para editar. Las tres por defecto **no se pueden borrar** desde la ventana — son de las que parte todo lo demás.

Si algo falla

- **KeyError: 'custodian'** — esa instalación no está en `config/machine.yaml` . Es válido tener solo maestro y conductor.
- **FileNotFoundError del donante** — falta `AlgoData/projectsBackup/XAUUSD_base_2026-09-21/project.cfx` . Es el donante congelado del que clona todo; si no está, hay que recuperarlo antes de seguir.
- **StopIteration en builder_roles** — el `.cfx` que le has pasado con `--donor` no tiene tarea Build, y los papeles solo viven ahí.
- **'<slug>' already exists** al clonar — ya hay una paleta con ese nombre de fichero. Dale otro nombre; no se sobrescribe una paleta por accidente.
- **FileNotFoundError de un .yaml de paleta** — le has pedido una paleta que no está en la librería. `python3 -m sqx.blocks.palette` sin argumentos dice cuáles hay.

22. Plantillas — de una idea a una plantilla que construye

Qué pregunta responde

Tienes una idea de entrada —"que el cierre cruce por encima de la banda superior de Keltner"— y quieres una plantilla de AlgoWizard que la construya. Estas tres herramientas hacen el camino entero sin abrir la GUI: autoran el bloque que falte, lo meten en la instalación, montan la plantilla y anotan lo que se ha probado y dónde.

Cuándo lo usas, y cuándo no

Cuando la lógica esté decidida: qué condición exactamente, en qué dirección, qué se fija y qué se deja aleatorio. Eso lo decide el brief, y el brief lo aprueba el dueño.

No sirve para decidir si la idea es buena, ni sustituye a una construcción real: lo que produce es una plantilla que SQX acepta. Si la estrategia gana dinero es otra pregunta y otras herramientas.

Antes de empezar

1. **La instalación de destino tiene que estar parada.** `customBlocks.xml` es el almacén de verdad, y una GUI abierta lo reescribe al salir: tu escritura se pierde en silencio. `sqx/blocks/install.py` se niega a escribir si el puerto de esa instalación contesta.
2. **Regla dura 5:** `python3 -m core.assets <SÍMBOLO>` antes de tocar cualquier plantilla o tarea. Si sale distinto de cero, se para y se pregunta.
3. **El bloque tiene que estar en las dos instalaciones** si autoras en el conductor y construyes en el custodio. `python3 -m sqx.inspect.vocabulary --diff custodian` te lo dice en un segundo.

Cómo se ejecuta

```
# 1. meter los bloques autorados en una instalación parada
python3 -m sqx.blocks.install ~/Desktop/AlgoData/templates/library/<nombre>/deps/blocks.xml --
role conductor
python3 -m sqx.blocks.install ~/Desktop/AlgoData/templates/library/<nombre>/deps/blocks.xml --
role custodian

# 2. montar la plantilla fijando un bloque en el esqueleto
python3 -m sqx.templates.build <nombre> <deps/blocks.xml> <CBlock_clave> <salida.sqx> [--shape
market_long] [--install custodian]

# 3. anotar lo que existe y lo que se ha probado
python3 -m sqx.templates.registry --set name=<nombre> --set archetype=breakout ...
python3 -m sqx.templates.registry --run --set template=<nombre> --set symbol=XAUUSD --set
timeframe=H1 ...
```

flag	obligatorio	qué hace
<code>install.py --role</code>	no	<code>conductor</code> (por defecto) o <code>custodian</code> . El maestro está prohibido
<code>build.py --shape</code>	no	esqueleto a transplantar; por defecto <code>market_long</code>
<code>build.py --install ROLE</code>	no	además copia el <code>.sqx</code> a <code>StrategyTemplates/</code> de esa instalación
<code>build.py --param '#Type#=1'</code>	no	fija un parámetro que nombraste (una EMA: <code>#Type#=1</code>); el resto de números — periodos, desviaciones— los sortea el builder en cada estrategia. Repetible

Los periodos son aleatorios, siempre salvo que des un valor (dueño, 2026-09-26). SQX no deja sortear los parámetros de un bloque fijo, así que `build.py` mete tu condición en un **grupo de un solo ítem** (`<nombre>Signal`) y ata a él el primer hueco: todas las estrategias llevan tu condición, cada una con su periodo. El grupo se escribe también en `deps/groups.xml`, y hay que instalarlo en los dos workers con `python3 -m sqx.blocks.install deps/groups.xml --role conductor` (y `--role custodian`). Comprobado 2026-09-26 en USDJPY H1: 15 periodos distintos de la EMA en 20 estrategias.

Un bloque nativo (no autorado) se fija igual, pasando en lugar de `deps/blocks.xml` el `config.xml` de AlgoWizard del conductor. Salida real, 2026-09-25, que reproduce la `emaCloseAbove` que se hizo a mano y ya está confirmada en build:

```
$ python3 -m sqx.templates.build emaCloseAbove \
~/Desktop/SQX_w1/internal/web/SQWIZARD/branding/global/config.xml MABarClosesAbove \
template.sqx --param '#Type#=1'
template.sqx (5343 bytes, shape market_long, fixed MABarClosesAbove)
```

| `build.py --set` | no | subcarpeta usada al instalar; por defecto `authored` || `registry.py --run` | no | escribe en `runs.csv` en vez de en `registry.csv` || `registry.py --set COL=VALOR` | sí | un valor por columna, repetido tantas veces como columnas |

Los tres son instantáneos. Ninguno arranca SQX; `install.py` exige que esté parado.

Qué produce

<code><instalación>/user/settings/customBlocks.xml</code>	modificado, con copia fechada al lado
<code><instalación>/user/settings/StrategyTemplates/<set>/</code>	la plantilla, si usas <code>--install</code>
<code>AlgoData/templates/library/<nombre>/template.sqx</code>	la plantilla
<code>AlgoData/templates/registry.csv</code>	una fila por plantilla
<code>AlgoData/templates/runs.csv</code>	una fila por plantilla × símbolo ×
<code>timeframe</code>	

`install.py` **reemplaza** un bloque cuya clave ya exista en vez de añadirlo dos veces: SQX lee la primera coincidencia y un duplicado no se ve hasta que una construcción usa el equivocado. Antes de escribir deja una copia fechada en `customBlocks-backups/`, igual que hace la GUI.

Cómo se lee el resultado

```
$ python3 -m sqx.blocks.install ../deps/blocks.xml --role conductor
SQX_w1: added ['CBlock_CloseCrossesAboveKCUpper', 'CBlock_CloseCrossesBelowKCLower'], replaced
[]
store now holds 173 blocks; previous copy kept at ../customBlocks-backups/1790090019825.xml

$ python3 -m sqx.inspect.vocabulary --diff custodian
SQX_w2 lacks, against SQX_w1:
custom blocks (2): CBlock_CloseCrossesAboveKCUpper, CBlock_CloseCrossesBelowKCLower
```

Esa segunda salida es la que hay que mirar siempre: el bloque estaba en el conductor y **no** en el custodio, que es donde se construye. Una plantilla que referencia un bloque ausente no da error — se queda coja y desaparece del builder.

```
$ python3 -m sqx.templates.build keltnerUpperCrossUp deps/blocks.xml
CBlock_CloseCrossesAboveKCUpper template.sqx
.../template.sqx (5267 bytes, shape market_long, fixed CBlock_CloseCrossesAboveKCUpper)
```

`build.py` aborta si el bloque no aterriza en la señal, si queda más de un hueco aleatorio, o si el XML resultante no parsea. Los tres fallaron durante el desarrollo, así que los tres se comprueban.

Configurar el proyecto desde `assets/`

Un proyecto se clona de un donante, y **el donante lleva los ajustes vivos del maestro**, que no son la política declarada. En XAUUSD el donante cobra `SizeBased 8` donde `assets/` dice `PercentageBased 0.00174`, y cero slippage donde `assets/` dice cinco puntos. Si nadie lo pisa, la construcción corre a un coste que no ha elegido nadie, y no salta ningún error.

```
python3 -m sqx.projects.configure <ruta/project.cfx> XAUUSD
```

Sin `--segment`, **cada tarea toma el suyo de su tipo**: `Build` → `build`, todo lo demás → `oos1`, según `assets/_policy.yaml`. Eso es lo que hace útil declarar `spread_is` y `spread_oos` por separado; forzar un solo tramo en toda la cadena valora los retests con el spread de construcción.

```
cfgtest.cfx XAUUSD
build: spread 10.0, slippage 5, PercentageBased 0.00174, swap percent -7/-7,
      ventana 1199145600000 a 1514764800000
1 tarea(s): Build-Task3.xml
oos1: spread 10.0, slippage 5, PercentageBased 0.00174, swap percent -7/-7,
      ventana 1514764800000 a 1672531200000
14 tarea(s): Retest-Task1.xml, ... Retest-Task14.xml
△ PROVISIONAL: spread_is, spread_oos, commission, slippage, swap_long, swap_short
△ rangos MC Retest sin decidir: spread, slippage
```

Se niega ante tres cosas: `oos2` (sólo si lo pide un agente autónomo, `ALGO_AUTONOMOUS=1`), un coste con `use: null`, y un `.cfx` que tenga abierto una instalación viva, porque lo reescribe al salir.

Un ejemplo completo

La cadena entera del `keltnerUpperCrossUp`, tal como se corrió el 2026-09-22:

```
python3 -m core.assets XAUUSD # regla dura 5 → exit 0
python3 -m sqx.blocks.install ../deps/blocks.xml --role conductor
python3 -m sqx.blocks.install ../deps/blocks.xml --role custodian
python3 -m sqx.templates.build keltnerUpperCrossUp ../deps/blocks.xml \
    CBlock_CloseCrossesAboveKCUpper ../template.sqx
# proyecto de una sola tarea sobre el donante, con la plantilla apuntada
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=start
name=smoke_keltnerUpperCrossUp','custodian')"
```

Resultado, leído del `-project action=status`:

Strategies generated	13639
Rejected	99.48 %
Running time so far	29 s.
In databank	30

Y la comprobación que decide si la plantilla sirve de algo — que las estrategias construidas lleven **de verdad** el bloque fijo:

```
estrategias: 30    llevan el bloque fijo: 30
companeros que eligio el builder: CSSAMarketRegimeAboveLevel x15, VWAP x5,
                                   BollingerBands x5, IsFalling x4, HighD x3, UlcerIndex x2, ...
```

30 de 30 con el Keltner fijo, cada una con un compañero aleatorio distinto. Eso es exactamente lo que se pidió: una condición fija y una libre.

Qué NO te dice

- **No dice si la estrategia es rentable.** Una construcción de humo prueba la cadena, no la idea. 30 estrategias con tope de 30 no son una muestra de nada.
- **No prueba que el bloque calcule lo que su nombre dice.** Que SQX lo acepte y construya con él no significa que el cruce esté bien definido. Eso solo lo dice leer sus trades.
- **No valida los costes.** El spread, la comisión y el point value de XAUUSD son a día de hoy los defaults de SQX, provisionales, no cifras acordadas con el bróker.
- **registry.csv no se lee solo.** Anota lo que le digas; si no anotas una corrida, la respuesta a "¿ya lo probé?" será que no.

Si algo falla

`SQX_w1 is running on port 5060` — la instalación está levantada y reescribiría tu cambio al salir. Párala: `bin/sqx-worker.sh --role conductor stop`.

`CBlock_X did not land in the signal` — la clave no está en el XML de bloques que le pasaste, o el esqueleto no tiene dos huecos aleatorios.

La construcción termina y el databank está vacío en disco. No es un fallo: el databank está en `Auto-sync never` y SQX guarda en memoria. Se arregla parando la instalación, poniendo ese databank en `Auto-sync every 1 hour` dentro del `project.cfx` —nunca con una instancia abierta, regla dura 4— y reconstruyendo. Está contado en `knowhow/databanks/memory-vs-disk-exporter.md`.

El proyecto arranca y no testea nada. Estás usando `startOnlyTask`, que informa de éxito y no hace nada. Usa `action=start`, y un `action=stop` antes de cada segundo arranque.

28. Builder — de una plantilla a un proyecto que arranca, en un comando

Qué pregunta responde

Tienes una plantilla en la librería y quieres que SQX genere estrategias con ella sobre un activo. Esto monta el proyecto entero: clona el donante, se queda solo con la tarea de construcción, apunta la plantilla, pone los topes, deja los databanks escribiendo a disco, fecha la ventana desde `assets/` y lo instala. **Hasta el 2026-09-23 esos cinco pasos se editaban a mano en el XML de la tarea**, que es lo que impedía meter la cadena dentro de una aplicación.

Cuándo lo usas, y cuándo no

Cuando la plantilla ya existe y está registrada. No sirve para crear la plantilla —eso es `/sqx-strategy-template`— ni para lanzar la construcción: esto deja el proyecto listo, y arrancarlo es una decisión aparte porque cuesta CPU.

Antes de empezar

1. **La instalación de destino tiene que estar parada.** Reescribe el `project.cfx` al salir y se comería el proyecto recién escrito (regla dura 4). El comando se niega si el puerto contesta.
2. **Regla dura 5:** el activo tiene que tener sus costes pactados. Si alguno sigue en `use: null`, el comando para y te dice cuál.
3. **El bloque de la plantilla tiene que estar en esa instalación:** `python3 -m sqx.inspect.vocabulary --diff custodian`.

Cómo se ejecuta

```
python3 -m sqx.projects.builder Test_chat_v2 --purpose "probar la plantilla keltner" \
  --template ~/Desktop/AlgoData/templates/library/keltnerUpperCrossUp/template.sqx \
  --symbol XAUUSD --role custodian --max-strategies 25 --minutes 8
```

flag	obligatorio	qué hace
<code>name</code>	sí	<code>Test_<...></code> para una prueba de funcionamiento que se tira al acabar, <code>Trade_<...></code> para un proyecto que tiene que dar fruto de verdad; solo guiones bajos — la API parte por espacios. Cualquier otro nombre se rechaza
<code>--purpose</code>	sí	una frase: para qué es el proyecto. Queda en <code>AlgoData/projects/registry.csv</code> , y es lo que lee quien encuentre el proyecto semanas después
<code>--template</code>	sí	el <code>.sqx</code> de la librería
<code>--symbol</code>	sí	activo de <code>assets/</code> ; de ahí salen la ventana, la sesión, el feed y los costes
<code>--role</code>	no	instalación destino; <code>custodian</code> por defecto
<code>--max-strategies</code>	no	tope de estrategias, 30 por defecto
<code>--minutes</code>	no	tope de reloj, 10 por defecto
<code>--donor</code>	no	proyecto del que clona; el XAUUSD congelado por defecto
<code>--session-from</code>	no	un <code>project.cfx</code> que fuerce de dónde tomar la sesión y el feed del activo, si no quieres el que el comando encontraría solo. Sólo lectura , nunca se escribe en él
<code>--json</code>	no	emite el resultado como JSON y nada más — esto es lo que consume un chat

Instantáneo: mueve ficheros y reescribe XML, no arranca SQX.

Encadena los databanks solo. El input de cada tarea pasa a ser el output de la anterior, en el orden que lleva `config.xml`. Hace falta porque al quedarte con un subconjunto de tareas (`--only`) las supervivientes siguen apuntando a donde apuntaban en el donante: sin esto, la tarea de mercados adicionales leía **su propio output** y el cross-check corría sobre un databank vacío. `--json` lo devuelve en `chain`, con lo que cada tarea leía antes. La primera tarea se queda con el input del donante a propósito — no hay nada anterior que darle, y un Build genético se siembra de `Initial population`, no de su `Input`.

Otros activos además de XAUUSD — arreglado el 2026-09-27

Hasta el 2026-09-23 `--symbol` no cambiaba el mercado: aplicaba la doctrina, el timeframe, la sesión y la ventana del activo que nombraras, pero el feed seguía siendo el del donante (oro por defecto), así que un `--symbol USDJPY` construía en `XAUUSD_DukasM1_Infinox` al spread del oro, con el horario de USDJPY y sin que `USDJPY_DukasM1_the5ers` apareciera en ninguna tarea.

Desde el 2026-09-27 (`sqx/projects/source.py`) el comando **también sustituye el feed**: busca, en cualquier instalación, el `project.cfx` más reciente que defina a la vez la sesión y el `<Symbol>` del activo, y copia los dos —nunca solo la sesión, que era la trampa original (`knowhow/costs/sessions-per-asset.md`) —. El resumen dice de qué proyecto los tomó, por ejemplo:

```
feed      XAUUSD_DukasM1_Infinox → USDJPY_DukasM1_the5ers en 18 tarea(s), traído de
          .../Test_USDJPY_mcrRanges/project.cfx
sesión    USDJPY_ftmo añadida a 17 tarea(s) que la nombraban sin definirla
```

Comprobado de punta a punta el 2026-09-27 (`Test_USDJPY_donchianUpperCrossUp_M30`, sin `--session-from`): 200/200 estrategias construidas sobre `USDJPY_DukasM1_the5ers`, y el workflow completo — puerta, mercados adicionales, crossTF, MC Retest, SPP, variantes, WFC y estructura — leyó ese mismo feed en

cada paso.

Si el activo no tiene ningún proyecto que defina su sesión y su feed (medido: EURUSD, AUDUSD, USDCAD, GBPUSD hoy), el comando se niega en vez de inventar nada — regístralo en SQX primero, o pasa `--session-from` a mano si sabes de qué proyecto tomarlo.

Antes de fiarte de un proyecto para un activo nuevo, míralo:

```
python3 -c "  
import zipfile, re  
z = zipfile.ZipFile('<instalación>/user/projects/<nombre>/project.cfx')  
for n in sorted(z.namelist()):  
    if n == 'config.xml': continue  
    t = z.read(n).decode('utf-8', 'replace')  
    print(n, sorted(set(re.findall(r'<Chart symbol=\"([^\"]+)\", t))))  
"
```

Si el feed de tu activo no sale ahí, el proyecto no es de tu activo. `OPEN.md` issue 35 tiene la causa y las dos vías de arreglo.

Qué produce

<code><instalación>/user/projects/<nombre>/project.cfx</code>	el proyecto
<code><instalación>/user/settings/StrategyTemplates/authored/</code>	la plantilla, con su propio nombre

La plantilla se instala con el nombre de **su carpeta de librería**, no con el del fichero: todas las carpetas guardan un `template.sqx`, así que instalarlas por el nombre del fichero haría que dos plantillas distintas se pisaran en silencio.

Cómo se lee el resultado

```
chat_v2 on SQX_w2  
  template  ../StrategyTemplates/authored/keltnerUpperCrossUp.sqx  
  tasks     Build   caps 25 strategies / 8 min  
  segments  Build-Task3.xml=build  
  to disk   Results, Last generation, Initial population, Strategies to improve  
  costes    NO van en la tarea: una InstrumentInfo que no coincida con el registro  
            de SQX deja el proyecto irresoluble. Aplícalos con el install vivo:  
            -instrument action=edit instrument=XAUUSD_Infinox defaultspread=10.0 ...  
  ▲ PROVISIONAL: spread_is, spread_oos, commission, slippage, swap_long, swap_short
```

Cuatro cosas que decide esa salida:

- **to disk** — los databanks que estaban en `Auto-sync never` y ahora escriben. Sin eso, tras generar el directorio queda vacío y `/curate` no tiene ficheros que mover.
- **segments** — de qué tramo sale la ventana de cada tarea. `build` es la única muestra que ve el generador.

- **costes** — no se escriben en la tarea, y el motivo está medido: ver `knowhow/costs/per-task-costs.md`.
- **Los rangos del MC Retest sí se escriben**, porque viven en los `<Method type="Randomize*>` de la propia tarea y no en la `InstrumentInfo`. Salen de `mc_retest:` del activo; los que sigan en `null` se saltan en vez de inventarse, y la preflight lo avisa.
- **PROVISIONAL** — cifras de trabajo del dueño, no pactadas con el bróker. Marcan todo resultado.

Si la plantilla fuese a ignorarse — `type="simple"` con un `templateFile` puesto— **aborta** en vez de construir. Esa comprobación es gratis: lee un atributo, mientras que demostrar lo mismo después cuesta la construcción entera.

Un ejemplo completo

```
python3 -m sqx.projects.builder Test_chat_v2 --purpose "humo" --template
.../keltnerUpperCrossUp/template.sqx \
  --symbol XAUUSD --role custodian --max-strategies 25 --minutes 8
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; print(worker.call('-project action=start
name=chat_v2','custodian'))"
```

Resultado real del 2026-09-23: el proyecto arranca, genera **25 estrategias**, quedan **25 en disco**, y la puerta de la plantilla da **25/25 llevan el bloque fijo** (`python3 -m sqx.inspect.template_check --role custodian chat_v2`).

Qué NO te dice

- **No dice si la estrategia sirve.** Un proyecto que arranca y genera no es una estrategia buena.
- **No aplica los costes.** Los imprime como comando; aplicarlos en el registro de SQX es un paso aparte y **global a la instalación**.
- **No lanza la construcción.** A propósito: eso cuesta horas y es una decisión del dueño.

Todo el workflow en un solo proyecto: `--workflow`

Desde el 2026-09-25 (decisión del dueño), cuando una plantilla entra en el workflow el proyecto se crea con `--workflow`: lleva **todas** las tareas de los pasos siguientes —mercados, crossTF, MC Retest, SPP, WFC y WFM— apagadas salvo `CONSTRUCCION` y `00S`, e ignora `--tasks` y `--only`. Cada paso enciende sólo las suyas antes de lanzar. Todo el detalle en `04-sqx-plantillas-y-proyectos.pdf` > `47-proyecto-workflow`.

Si algo falla

`Project has unresolved resources` al arrancar — la `InstrumentInfo` de alguna tarea no coincide con el registro de SQX. No lo arregla este comando; ver `knowhow/costs/per-task-costs.md`.

`the custodian is running and rewrites a project.cfx on exit` — párala primero.

`'<nombre>': project names are underscores only` — la API HTTP parte el comando por espacios.

La doctrina de construcción (`assets/_build.yaml`)

Desde el 2026-09-23 el builder no hereda nada del donante en lo que a la forma de la estrategia se refiere. Lo escribe todo desde `assets/_build.yaml` , y **en todas las tareas del proyecto**:

regla	valor por defecto
condiciones de entrada / de salida	como máximo 2 y 2
global lookback	1 barra (<code>minShift = maxShift = 1</code>)
tipos de orden	sólo a mercado
salidas	por barras o por condición; nada de SL, TP, trailing ni break-even
barras en mercado	2 a 24 horas , convertidas a barras según el timeframe
money management	<code>ATRRiskBasedSizingFixedRisk</code> , <code>ATR(20) × 4</code> , riesgo 1000 sobre 100.000
salir el viernes	sí, a las 21:00
motor	MetaTrader5 (hedged)
databank	10.000 estrategias, parada por databank lleno
crosschecks	sólo el de alta precisión, al mismo spread que el test principal
genético	copia verbatim de <code>XAUUSD_Breakout_H1</code>

El timeframe es obligatorio y va en `--timeframe` . Todas las tareas lo comparten: un retest en otro timeframe no está probando la estrategia que se construyó.

```
python3 -m sqx.projects.builder Test_XAU_doctrina --purpose "ver la doctrina aplicada" --
timeframe M30 \
    --template ~/Desktop/AlgoData/templates/library/keltnerUpperCrossUp/template.sqx \
    --symbol XAUUSD --role custodian --tasks Build,Retest
```

doctrina M30 en todas las tareas, sesión XAUUSD_the5ers, salida por barras 4–48
sesión XAUUSD_the5ers añadida a 14 tarea(s) que la nombraban sin definirla

Comprobado sobre las 20 estrategias que generó `algo_XAU_doctrina_smoke` : **20 de 20** con exactamente 2 condiciones de entrada, 1 de salida, `EnterAtMarket` y nada más, shift 1 en los 31 sitios donde aparece, y cero stop loss, profit target o trailing.

Quedarse con una sola tarea de retest

El donante lleva catorce. `--tasks Retest` las conserva todas, que es la cadena entera; para un proyecto de dos pasos —construir y cruzar la puerta del OOS— se nombra la que se quiere:

```
python3 -m sqx.projects.builder Test_XAU_IS00S_ejemplo --purpose "ejemplo IS/00S" --timeframe
M30 --symbol XAUUSD \
    --template <plantilla> --role custodian \
    --tasks Build,Retest --only Build-Task3.xml,Retest-Task1.xml
```

`Retest-Task1.xml` es la que lee de `Results` y escribe en `00S` . Las demás son la cadena de Monte Carlo, los SPP y la WFM, cada una encadenada al databank de la anterior.

Hay un ejemplo construido y corrido, `XAU_IS00S_ejemplo` en el custodio: cómo se hizo y qué tiene en `knowhow/locations/xau-isoos-example-project.md` .

47. El proyecto del workflow — todas las pruebas de una estrategia en un solo custom project

Qué pregunta responde

¿Dónde corre cada prueba de SQX de una población, del build a la Walk Forward Matrix? En **un único custom project**, creado en el paso 5 con todas las tareas del workflow dentro. Cada paso enciende sólo las suyas antes de lanzar, y SQX se salta las apagadas. Dos comandos: `builder --workflow` lo crea y `stage` elige qué paso corre en el próximo `start`.

Decisión del dueño, 2026-09-25: "que se creen todas las tasks dentro del mismo custom project".

Cuándo lo usas, y cuándo no

- **Sí:** siempre que una plantilla entra en el workflow. `/template-run` lo crea así, y el resto de skills (`/crossmarket`, `/crosstf`, `/mcretest`, `/spp`, `/variants`, `/wfm`) configuran y corren su tarea dentro de ese mismo proyecto.
- **No:** para una prueba suelta sobre un proyecto hecho sólo para ella (por ejemplo el antiguo `USDJPY_variantes`). Ahí se sigue usando `builder --tasks ... --only ...`. `stage` se niega sobre un proyecto que no lleva las tareas del workflow, y los configuradores lo avisan con un `△`.

Antes de empezar

- `python3 -m core.assets <SIMBOL0>` con salida 0 (regla dura 5).
- **La instalación donde vive el proyecto, parada.** SQX reescribe el `.cfx` al salir y se perdería el cambio (regla dura 4). Los dos comandos se niegan si la instalación está arriba.
- Una plantilla ya en la librería (`~/Desktop/AlgoData/templates/library/<nombre>/template.sqx`).

Cómo se ejecuta

Crear el proyecto (paso 5):

```
python3 -m sqx.projects.builder Trade_XAUUSD_keltner_H1 --purpose "workflow entero de la
keltner en H1" \
  --template ~/Desktop/AlgoData/templates/library/keltnerUpperCrossUp/template.sqx \
  --symbol XAUUSD --timeframe H1 --role custodian --workflow --max-strategies 2000 --minutes
90
```

Elegir qué corre en el próximo `start`, a mano:

```
python3 -m sqx.projects.stage --cfx
~/Desktop/SQX_w2/user/projects/XAUUSD_keltner_H1/project.cfx \
  --step spp
```

flag	obligatorio	qué hace
<code>--workflow</code> (builder)	no	mete todas las tareas del workflow en el proyecto; sin él, el builder se queda con <code>--tasks</code> / <code>--only</code> como antes
<code>--cfx</code> (stage)	sí	el <code>project.cfx</code> del proyecto
<code>--step</code> (stage)	sí	uno o varios pasos separados por comas: <code>build</code> , <code>oos</code> , <code>crossmarket</code> , <code>crosstf</code> , <code>mcretest</code> , <code>spp</code> , <code>wfm</code> , <code>wfc</code>

Normalmente no hace falta llamar a `stage` : **cada configurador lo hace solo para su paso** al terminar (`sqx.projects.crossmarket` , `crosstf` , `mcretest` , `spp` , `wfc` , `wfm`). Tardan menos de un segundo y no arrancan SQX.

Qué produce

<instalación>/user/projects/<nombre>/project.cfx con 18 tareas:

paso	tarea(s)	lee → escribe
build	CONSTRUCCION	— → Results
oos	OOS	Results → OOS
crossmarket	Retest Markets - Family	OOS → Retest Markets - Family
mcretest	MCR 1 Bar ... MCR 8 Stress	lo que diga --input
spp	SPP IS , SPP OOS	lo que diga --input
wfm	WFM	lo que diga --input
crosstf	CrossTF (añadida)	CrossTF_Input → CrossTF , ventana build..oos1
wfc	WFC 1 IS , WFC 2 OOS1 , WFC 3 OOS2 (añadidas)	WFC_Variants → WFC_Build , WFC_OOS1 , WFC_OOS2

MC Trades y las tareas auxiliares del donante (Clear databanks , Go To Task) no entran: ningún paso las corre, y un Go To Task activo hace que `action=start` no acabe nunca. **Sobrescribe** un proyecto que ya tenga ese nombre en esa instalación.

Cómo se lee el resultado

Salida real del builder, 2026-09-25 (la línea nueva es `workflow`):

```
test_workflow on SQX_w1
  template ~/Desktop/SQX_w1/user/settings/StrategyTemplates/authored/keltnerUpperCrossUp.sqx
  workflow + CrossTF, WFC 1 IS, WFC 2 OOS1, WFC 3 OOS2: todas las tareas del workflow, activas
sólo CONSTRUCCION y OOS. CrossTF va a build..oos1; las WFC se precian con sqx.projects.wfc
  doctrina H1 en todas las tareas, sesión XAUUSD_ftmo, salida por barras 2-24
  to disk Results, Last generation, Initial population, Strategies to improve
  △ PROVISIONAL: spread_is, spread_oos, commission, slippage_is, slippage_oos, swap_long,
swap_short
```

Salida real de `stage --step spp` sobre ese proyecto. ● corre en el próximo `start` , · se salta:

```
test_workflow: el proximo `action=start` corre spp
  · CONSTRUCCION
  · 00S
  · Retest Markets - Family
  · MCR 1 Bar
  ...
  · MCR 8 Stress
  ● SPP IS
  ● SPP 00S
  · WFM
  · CrossTF
  · WFC 1 IS
  · WFC 2 00S1
  · WFC 3 00S2
```

Lo que dice el log de SQX al lanzar el paso `wfc` (☐ 2026-09-25, conductor, 3 madres):

```
CONSTRUCCION : SKIPPED, inactive task
...
CrossTF : SKIPPED, inactive task
WFC 1 IS : Task finished in 16.52 s.
WFC 2 00S1 : Task finished in 7.03 s.
WFC 3 00S2 : Task finished in 5.60 s.
Project finished
```

Lo que hay que mirar: que las ● sean justo las del paso que vas a correr. Si `CONSTRUCCION` sale con ● cuando no toca, el `start` vuelve a construir desde cero y pisa `Results` .

Un ejemplo completo

Un paso cualquiera, el SPP, sobre el proyecto del workflow en el custodio:

```
python3 -m core.assets XAUUSD
python3 -m sqx.projects.spp XAUUSD --cfx
~/Desktop/SQX_w2/user/projects/XAUUSD_keltner_H1/project.cfx \
    --input 00S                                # el databank con los supervivientes que llegan aquí
# ... imprime `el proximo `action=start` de XAUUSD_keltner_H1 corre solo `spp`
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=stop
name=XAUUSD_keltner_H1','custodian')"
python3 -c "from core import worker; worker.call('-project action=start
name=XAUUSD_keltner_H1','custodian')"
# mientras corre, sólo `-project action=status`; el final es `Project finished` en el log de
SQX
bin/sqx-worker.sh --role custodian stop
```

El `stop` antes del `start` es obligatorio: en un proyecto que ya corrió, un segundo `start` sin él no hace nada, y no avisa.

Qué NO te dice

- Que la tarea esté bien configurada: eso lo dice cada configurador en su salida. `stage` sólo enciende y apaga.
- Qué databank lee cada paso de la mitad del embudo: las MCR, el SPP y la WFM leen lo que les pases con `--input`, y eso lo decides tú según lo que haya sobrevivido.
- Nada sobre el `oos2`: la WFM y el WFC lo leen igual en un proyecto que en otro.

Si algo falla

- `<proyecto>` no lleva la(s) tarea(s) ... — no es un proyecto de workflow. Créalo con `--workflow` o usa el proyecto suelto sabiendo qué tareas tiene activas.
- el custodian tiene este proyecto abierto... — para la instalación con `bin/sqx-worker.sh --role custodian stop` antes de tocar el `.cfx`.
- `<proyecto>` no es un proyecto de workflow (en un configurador) — ha escrito su tarea pero no ha tocado qué está activo. Míralo antes de lanzar.

6. Quitar estrategias descartadas — aplicar un veredicto dentro de SQX

Para **escribir** el veredicto con un filtro o con un nombre suelto: `04-sqx-plantillas-y-proyectos.pdf > 27-curar`. Esta página es el paso que lo aplica.

Qué pregunta responde

Ninguna. Este no analiza nada: **ejecuta una decisión que ya tomaste**. Coge el `verdict.csv` que produjo un análisis y saca de SQX las estrategias marcadas como `DESCARTAR`, para que la databank de trabajo quede sólo con las que siguen vivas. **Las descartadas se borran**; antes queda anotado qué había, con la identidad y las métricas de cada una. Con `--into` en vez de borrarlas van a otra databank del mismo proyecto, y entonces SQX las vuelve a cargar al arrancar: cuestan RAM.

Es el único comando del proyecto que **cambia lo que contiene una databank**. Cada `.sqx` pesa unos 5 MB; guardar 8.000 descartadas de 10.000 serían 40 GB por una estrategia que nadie va a recuperar. Se anota y se borra (decisión del dueño, 2026-09-23).

Cuándo lo usas, y cuándo no

Lo usas cuando ya has leído el informe, estás de acuerdo con el veredicto, y quieres que la databank refleje esa decisión sin ir estrategia por estrategia en la interfaz.

No lo usas para "limpiar" una databank por criterios que no estén en un `verdict.csv`. El fichero es la traza de por qué se movió cada una; sin él no queda constancia de nada.

No lo usas con la interfaz de SQX abierta. Se niega a funcionar, y hace bien: las bases de datos del master están bloqueadas mientras la interfaz las tiene, y un cambio hecho por debajo de una instancia en marcha se pierde en silencio en la siguiente sincronización.

Antes de empezar

1. **Para la instalación.** `--role master` exige la interfaz cerrada; `--role conductor` o `--role custodian`, el worker parado (`bin/sqx-worker.sh --role custodian stop`). Se niega si está viva: SQX tiene los registros en memoria y reescribe los ficheros desde ella, así que un movimiento hecho por debajo se deshace en la siguiente sincronización.
2. **La databank de destino no hace falta crearla a mano.** Se crea sola si no existe, y el arranque siguiente la recoge — verificado en el custodio el 2026-09-23. La nota anterior, que decía que había que crearla desde la interfaz, no se sostiene.
3. Ten a mano la ruta del `verdict.csv`. Si trae la columna `identity` (la escribe `verdict.py` y cualquier módulo que pueda calcularla), el comando comprueba que el fichero que va a mover es la estrategia que se juzgó, y no otra que SQX haya puesto bajo el mismo nombre. Esa comprobación se hace ya en seco, con la instalación viva, porque sólo lee.

Actualizado el 2026-09-23. Antes esto llamaba a `-databank action=move` con la lista de nombres. **Ese selector no funciona:** por la API HTTP el nombre se corta en su primer espacio, y un `sqcli` de un disparo ni siquiera carga los registros — los dos informan de éxito y no hacen nada, y `move` sin selector se lleva la databank entera. Ahora mueve los ficheros `.sqx` con la instalación parada, y es el arranque siguiente el que hace que la memoria coincida. Medido y escrito en [knowhow/databanks/](#).

Cómo se ejecuta

Siempre dos veces: primero en seco, y sólo después de leer lo que sale, en serio.

```
# En seco. No toca absolutamente nada. Puedes lanzarlo con SQX abierto
python3 -m sqx.curate.apply_verdict --project XAUUSD --databank RetestMarkets \
    --verdict ~/Desktop/AlgoData/reports/XAUUSD/RetestMarkets/2026-09-
08/randomentry/verdict.csv \
    --role custodian

# En serio. Con la instalación parada
python3 -m sqx.curate.apply_verdict --project XAUUSD --databank RetestMarkets \
    --verdict ~/Desktop/AlgoData/reports/XAUUSD/RetestMarkets/2026-09-
08/randomentry/verdict.csv \
    --role custodian --apply
```

flag	obligatorio	qué hace
<code>--project</code>	sí	proyecto en esa instalación
<code>--databank</code>	sí	databank de la que salen
<code>--verdict</code>	sí	ruta del <code>verdict.csv</code>
<code>--into</code>	no	databank del mismo proyecto a la que moverlas en vez de borrarlas . SQX la carga al arrancar, así que cuestan RAM
<code>--role</code>	sí	la instalación que tiene el proyecto: <code>master</code> , <code>conductor</code> o <code>custodian</code> . No tiene valor por defecto: sin él el comando se niega, para que nunca se toque el maestro por olvido
<code>--apply</code>	no	sin él no se toca nada. Con él, se borra

Instantáneo: mueve ficheros, no arranca SQX. Lo que tarda es el arranque posterior, que es el que hace que la memoria coincida con lo que quedó en disco.

Y ese arranque es parte del procedimiento, no un detalle. Hasta que no lo hagas, SQX sigue teniendo en memoria lo que había antes:

```
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; print(worker.call('-databank action=list project=
<P>', 'custodian'))"
```

El recuento que imprime ahí es la única verificación que vale: es lo que va a leer la tarea siguiente.

Qué produce

Tres cosas, en este orden:

1. **El registro**, en `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/curate/`, junto al veredicto: `before-HHMMSS.csv` con nombre, identidad, tamaño, veredicto **y el motivo** de cada estrategia que había en disco, y `rejected-HHMMSS.csv` con la fila de métricas de cada descartada, con el motivo como primera columna, si la databank tiene métricas exportadas. El motivo es la columna `reason` del veredicto: el filtro que no pasó (`failed: sharpe_ratio_is > 0.93`), `dropped by name`, o el nombre del fichero de veredicto cuando el módulo que juzgó no escribió motivo — entonces ese fichero es la prueba. Kilobytes. Copiar los `.sqx` costaría 50 GB por corte con 10.000 estrategias.
2. **El borrado** de los `.sqx` descartados (o el movimiento, con `--into`).
3. **Un `manifest.json`** junto al registro, diciendo qué veredicto lo produjo y cuántas se quitaron.

Cómo se lee el resultado

En seco, te dice cuántas se quedan, cuántas se mueven, y en cuántas ha podido comprobar la identidad. Salida real del 2026-09-23 en el conductor:

```
SQX_w1 · Retester/Results: 66 strategies on disk
  the verdict drops 32, keeping 34
  identity checked on 32 of 32

dry run. Re-run with --apply, with the conductor stopped.
```

Si un fichero no es la estrategia juzgada, se para ahí y no mueve nada. Probado cambiando un fichero por otro bajo el mismo nombre:

```
x 1 carry a different strategy than the verdict judged: Strategy 11.10.85(1). The databank
  changed under that name since the verdict was written; re-export and judge again. Nothing was
moved.
```

En serio, las líneas que importan son estas:

```
metrics of the dropped kept in
/home/sergioguslw/Desktop/AlgoData/reports/Retester/Results/2026-09-23/curate/rejected-
073842.csv
  what was here is listed in /home/sergioguslw/Desktop/AlgoData/reports/Retester/Results/2026-
09-23/curate/before-073842.csv
Results: 34 → 33   deleted 1
start the conductor and the sync from files makes memory match: the next task reads 33.
```

Y tras arrancar, SQX contó **33**. Ese es el número que lee la tarea siguiente.

Los números se cuentan **volviendo a mirar los ficheros en disco**, no creyéndose la respuesta de SQX. Si no cuadran con lo que decía el veredicto, el comando se para y te dice contra qué lista comparar el directorio.

Qué NO te dice

- **No te dice que la decisión fuera buena.** Aplica un fichero. La calidad del veredicto es la del análisis que lo escribió.
- **No conserva las operaciones de las descartadas.** Con el fichero se van sus trades y su XML. Lo que queda de la población completa son sus métricas en `rejected-HHMMSS.csv`, que es lo que leen los estudios de población; un estudio sobre trades de una estrategia descartada ya no es posible. Es la contrapartida de no guardar 40 GB.

Si algo falla

- **"the master is running"** — la interfaz de SQX sigue abierta. Ciérrala del todo y repite.
- **Los números no cuadran al final** — el comando se para solo y no ejecuta nada más. Compara el directorio con `before-HHMMSS.csv` antes de tocar nada.
- **"N had no file"** con los números cuadrando — el veredicto nombraba una estrategia que ya no estaba, porque las métricas se exportaron antes de un corte anterior. Lo demás se aplicó bien; vuelve a exportar las métricas antes del siguiente veredicto. En seco ya lo avisa: `⚠ 1 named by the verdict are not here`.
- **"carry a different strategy than the verdict judged"** — la databank cambió bajo ese nombre desde que se escribió el veredicto. No se movió nada: vuelve a exportar las métricas, vuelve a juzgar, y aplica el veredicto nuevo.

27. Curar una databank — escribir el veredicto con un filtro o con un nombre

Qué pregunta responde

Ninguna: **ejecuta lo que tú decides**. Tienes una databank con miles de estrategias y un criterio en Python, o una estrategia concreta que has visto en tu tabla y quieres fuera. Esto convierte esa decisión en un `verdict.csv`, que `04-sqx-plantillas-y-proyectos.pdf` > `06-mover-estrategias` aplica dentro de SQX.

Dos modos: `--keep` con un filtro sobre las métricas exportadas, y `--drop` con uno o varios nombres.

Cuándo lo usas, y cuándo no

Lo usas entre dos tareas de SQX: la generación acabó, exportaste las métricas, y quieres que la tarea siguiente lea sólo las que pasan tu corte. O cuando en tu tabla ves una que no quieres y la apartas por su nombre. Se puede repetir tantas veces como cortes quieras: cada corte deja su propio `verdict-HHMMSS.csv`, y al aplicarlo, su propio registro de lo que quitó.

No lo usas como sustituto de un análisis. Un filtro sobre métricas es un corte, no una prueba de robustez; para eso están `05-cribado-oos.pdf` > `02-filtros`, `08-montecarlo.pdf` > `07-montecarlo` y compañía, que escriben su propio veredicto.

Antes de empezar

1. **Las métricas exportadas.** `verdict.py` juzga `~/Desktop/AlgoData/metrics/<proyecto>/<databank>/metrics.csv` y nada más: `python3 -m sqx.export.export_metrics --project P --databank Results`. Si el CSV es viejo, el veredicto es viejo.
2. **Los ficheros `.sqx` en disco** en la instalación que nombras con `--role`: de ahí lee la identidad de cada estrategia. Una databank en `Auto-sync never` tiene el directorio vacío; el comando lo avisa (`identity from file on 0 of N`) y el paso siguiente se negará.
3. **No hace falta parar nada:** sólo lee. Para aplicar el veredicto sí (`04-sqx-plantillas-y-proyectos.pdf` > `06-mover-estrategias`).

Cómo se ejecuta

```
# qué columnas puedes usar en el filtro
python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor --columns

# un filtro: lo que NO lo cumple se marca DESCARTAR
python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor \
    --keep "sharpe_ratio_is > 0.9"

# una estrategia concreta, por su nombre tal y como lo muestra SQX
python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor \
    --drop "Strategy 10.16.68"
```

flag	obligatorio	qué hace
<code>--project</code>	sí	proyecto tal y como aparece en SQX
<code>--databank</code>	sí	databank que se juzga
<code>--role</code>	no	instalación que tiene los ficheros: <code>master</code> (por defecto), <code>conductor</code> o <code>custodian</code>
<code>--keep</code>	uno de los tres	expresión de pandas sobre las columnas de <code>--columns</code> . <code>and</code> , <code>or</code> , <code>></code> , <code><=</code> , <code>==</code>
<code>--drop</code>	uno de los tres	uno o más nombres, entre comillas porque llevan espacio
<code>--columns</code>	uno de los tres	imprime los nombres de columna y no escribe nada

Los nombres de columna son los de la exportación en minúsculas y con guiones bajos: `Net profit (00S)` es `net_profit_oos`, `# of trades (IS)` es `n_of_trades_is`. Instantáneo; no toca SQX.

Qué produce

`~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/curate/verdict-HHMMSS.csv`, uno por ejecución, nunca sobrescribe. Columnas `strategy`, `verdict` (`MANTENER` o `DESCARTAR`), `reason` (el filtro que falló, o `dropped by name`) e `identity` (SHA-256 de la definición de la estrategia, leído del fichero).

Cómo se lee el resultado

Salida real del 2026-09-23 sobre el conductor:

```
$ python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor --keep
"sharpe_ratio_is > 0.9"
Retester/Results: 66 judged, 34 kept, 32 DESCARTAR
  identity from file on 66 of 66
(/home/sergioguslw/Desktop/SQX_w1/user/projects/Retester/databanks/Results)
/home/sergioguslw/Desktop/AlgoData/reports/Retester/Results/2026-09-23/curate/verdict-
072117.csv
apply it: python3 -m sqx.curate.apply_verdict --project Retester --databank Results --verdict
... --role conductor # then --apply, stopped
```

La primera línea son los números; la segunda te dice si la identidad se pudo leer de todos los ficheros (si no es `N of N`, la databank no está en disco); la última es el comando que sigue.

Desde el 2026-09-28, `apply_verdict` exige `--role`. Antes, si no lo ponías, apuntaba al **maestro**: el único comando que borra estrategias podía tocar tu install por olvidar un flag. Ahora falla con `the following arguments are required: --role` y hay que decir `conductor`, `custodian` o, a propósito, `master`. El `--role` de `verdict.py` (que solo lee) no cambia.

Desde la ventana no hace falta nada de esto: filtras el databank y pulsas «Continuar workflow» (capítulo 69), que escribe el veredicto, copia antes los `.sqx` a `AlgoData/projects/discards/`, llama al mismo `apply_verdict` con el worker del proyecto y lanza la tarea siguiente. Nunca sobre el maestro.

Un ejemplo completo

El ciclo entero, tal y como se corrió el 2026-09-23 sobre `Retester/Results` en el conductor:

```

$ python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor --drop
"Strategy 10.16.68"
Retester/Results: 34 judged, 33 kept, 1 DESCARTAR

$ bin/sqx-worker.sh --role conductor stop
$ python3 -m sqx.curate.apply_verdict --project Retester --databank Results --verdict
.../verdict-073842.csv --role conductor --apply
SQX_w1 · Retester/Results: 34 strategies on disk
  the verdict drops 1, keeping 33
  identity checked on 1 of 1
  metrics of the dropped kept in .../reports/Retester/Results/2026-09-23/curate/rejected-
073842.csv
  what was here is listed in .../reports/Retester/Results/2026-09-23/curate/before-073842.csv
Results: 34 → 33   deleted 1
start the conductor and the sync from files makes memory match: the next task reads 33.

$ bin/sqx-worker.sh --role conductor start      # SQX cargó 33 estrategias en Results

```

Qué NO te dice

- **No te dice que el corte sea bueno.** Un filtro sobre métricas dentro de muestra selecciona lo que mejor se ajustó al pasado; si eso sobrevive fuera de muestra lo responden los análisis, no esto.
- **No sabe nada de RAM ni de SQX.** El veredicto es un CSV. Quien saca las estrategias de la memoria de SQX es `apply_verdict` con la instalación parada y el arranque siguiente.

Si algo falla

- **no metrics export at ...** — no has exportado las métricas de esa databank, o la exportaste con otro nombre. `sqx.export.export_metrics` primero.
- **not in the export: Strategy X** — ese nombre no está en el CSV. Mira si lleva `(1)` al final: SQX renombra así cuando dos estrategias colisionan.
- **identity from file on 0 of N** — la databank no está en disco. `04-sqx-plantillas-y-proyectos.pdf` > `06-mover-estrategias`, sección «Antes de empezar», explica el `Auto-sync never`.
- **Un error de pandas al evaluar --keep** — una columna mal escrita. `--columns` te da los nombres exactos.

17. Pipeline — mete cien estrategias, vuelve al cabo de unos días y lee los veredictos

Qué pregunta responde

El estudio de robustez de una estrategia son siete pasos encadenados: leer su SPP, diseñar las variantes, fabricarlas, correrlas en SQX, recoger los resultados, medir si la superficie aguanta fuera de muestra, y dictar un veredicto. Hacer eso a mano para **una** estrategia es una tarde. Para las ~100 madres de un databank son días de máquina.

Este comando lo hace solo. Le dices el proyecto, te vas, y cuando vuelves cada estrategia tiene su veredicto escrito, y los gigas de variantes que hicieron falta para llegar a él ya no ocupan disco.

Y mientras corre **te deja mirar**: hay un fichero por estrategia que dice, en todo momento, en qué paso va, por qué porcentaje y qué está haciendo. No tienes que adivinar si se ha colgado.

Cuándo lo usas, y cuándo no

Úsalo cuando ya has corrido los **SPP en SQX** y los tienes exportados: ése es el punto de partida de la cadena, y es lo único que tienes que hacer tú a mano.

No lo uses para una sola estrategia que estás mirando con lupa. Para eso corre cada etapa por separado — están pensadas para poder lanzarse sueltas, y cada una te enseña su informe. El pipeline es para cuando no quieres mirar.

No es un sustituto de las etapas. No calcula nada por su cuenta: las llama en orden, apunta lo que pasa y se niega a seguir cuando falta algo. Si borras `pipeline/run.py`, todo lo demás sigue funcionando a mano.

⚠ **Hoy solo la primera etapa es de verdad.** Las otras cinco (diseño, fabricación, ejecución, recogida y walk forward correlation) están siendo programadas por otros; mientras tanto, la cadena corre con **marcadores de posición** que escriben cifras inventadas. Sirven para probar que el encadenado funciona, **no para decidir nada**. Todo lo que escriben lleva dentro la marca `"placeholder": true`. Cuando cada módulo real esté listo, se cambia **una línea** en `pipeline/recipe.yaml` y esa etapa pasa a ser la de verdad.

Antes de empezar

- **Los SPP, corridos y exportados.** Tiene que existir `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/spp/`. Si no: `bash python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"`
- **Sitio en disco.** Antes de cada estrategia el pipeline mira lo que pesa `AlgoData` y **se para si algo está por encima de su presupuesto**. Es apuesta: lo que tiene que frenar una corrida de días es el presupuesto, no el disco lleno — cuando el disco se llena, la etapa que estaba escribiendo ya ha dejado medio fichero a medias. Para ver cómo estás: `bash python3 -m perf.disk.report` Los techos están en `perf/config.yaml`, en `disk.budget_gb`.
- **No toca SQX directamente.** Las etapas que sí lo harán todavía no existen. Hoy puedes lanzarlo con la GUI abierta.

Cómo se ejecuta

```
python3 -m pipeline.run --project XAUUSD --databank SPP_IS
```

flag	obligatorio	qué hace
<code>--project</code>	sí	nombre del proyecto tal y como aparece en SQX
<code>--databank</code>	no	carpeta del export, con guiones bajos en vez de espacios . Por defecto <code>SPP_IS</code>
<code>--strategy</code>	no	se puede repetir. Por defecto, todas las estrategias del export
<code>--day</code>	no	fecha del export a leer. Por defecto, la más reciente
<code>--report-day</code>	no	carpeta de informes en la que escribe sppUltra. Por defecto, hoy. Si reanudas una corrida al día siguiente, pon aquí el día en que empezó , o no encontrará los informes que ya hizo
<code>--asset</code>	no	nombre en <code>assets/</code> . Por defecto, el del proyecto. De ahí sale el sello de costes provisionales
<code>--sweep</code>	no	borra los datos de variantes al terminar cada madre. Sin él no se borra nada

Puedes pararlo cuando quieras con Ctrl-C, o apagar el ordenador. Al volver a lanzarlo **continúa donde estaba**: se salta lo que ya estaba hecho y retoma la etapa que quedó a medias.

Qué produce

Una carpeta por estrategia en `~/Desktop/AlgoData/pipeline/<proyecto>/<estrategia>/` :

- **state.json** — el registro. En qué paso va, con qué porcentaje, qué dijo cada etapa y cuándo terminó. Pesa kilobytes y **no se borra nunca**, ni siquiera cuando se borran los datos que describe. Es lo que convierte el borrado en algo auditable en vez de en un agujero.
- **state.json** guarda además, de cada etapa, lo que esa etapa haya declarado: cuántas variantes salieron, cuántos canarios fallaron, y la **huella del diseño** (`brief_hash`) con la que se fabricaron. Esa huella es la que permite darse cuenta de que el diseño cambió por el camino.
- **verdict.json** — el veredicto, con los números en los que se apoya y, si sale `reject` , qué umbral falló y por qué existe ese umbral.
- Los ficheros que va dejando cada etapa.

El informe de la primera etapa va donde siempre:

`~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/` .

Cómo se lee el resultado

Esto es una corrida real sobre `Strategy 17.9.39` :

```

1 estrategias madre en XAUUSD/SPP_IS

[1/1] Strategy 17.9.39
  sppultra python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS --
strategy Strategy 17.9.39
  | Strategy 17.9.39          proceed n_eff= 3,381 max 14.64 vs nulo 5.11 vivos=7
congelados=1
  | -> /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21
design python3 -m pipeline.stubs.placeholder --stage design --work ../Strategy_17-9-39
  | PROGRESS 20 paso 1 de 5 (marcador de posición de design)
  | PROGRESS 40 paso 2 de 5 (marcador de posición de design)
  ...
verdict python3 -m pipeline.stages.verdict --work ../Strategy_17-9-39 --brief
../design_brief_Strategy_17-9-39.json
  | PROGRESS 20 leyendo los números de 4 umbrales
  | PROGRESS 60 13 cifras reunidas
  | PROGRESS 100 proceed
-> verdict

```

Cada línea con sangría es **una etapa**, y debajo va lo que esa etapa imprime. Las líneas `PROGRESS` son el porcentaje: van a parar al registro mientras la etapa corre, no al acabar.

Si quieres mirar desde otra terminal cómo va, sin tocar nada:

```
cat ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39/state.json
```

```

{ "strategy": "Strategy 17.9.39", "stage": "verdict",
  "stages": {
    "sppultra": { "progress": 100, "status": "-> ../reports/XAUUSD/SPP_IS/2026-09-21",
                  "done_at": "2026-09-21T18:31:13" },
    "build":    { "progress": 100, "status": "paso 5 de 5", "n": 5000, "bytes": 56500000,
                  "done_at": "2026-09-21T18:31:15" } },
  "costs_provisional": true }

```

progress nunca baja. Si lo ves parado, la etapa está tardando; si ves que retrocede, es un fallo y el programa se para solo.

costs_provisional: true quiere decir que los costes con los que se ha calculado todo son los que trae SQX por defecto, no los reales de tu bróker. Sale de `assets/symbols/XAUUSD.yaml`, y dejará de salir cuando pongas ahí las cifras buenas.

Y el veredicto, cuando falla:

```
{ "verdict": "reject",
  "failed": [ { "number": "wfc.rho", "value": 0.41, "limit": 0.9, "bound": "min",
                "why": "below this the in-sample surface does not order the out-of-sample one"
              } ] }
```

Los umbrales son tuyos, y cambiarlos es gratis

Están en `pipeline/config.yaml`, bajo `verdict.rules`. Cada regla es un número, un límite y la razón por la que ese límite existe.

No todo número con un límite es un umbral. Los canarios, por ejemplo, no están ahí: un canario fallido no quiere decir que la estrategia sea mala, quiere decir que la medición no vale. Eso es una **puerta**, vive en `pipeline/recipe.yaml` y para la corrida, en vez de producir un veredicto que nadie debería leer. La diferencia es siempre la misma pregunta: ¿el número dice *esta estrategia no da la talla*, o dice *esta medición no significa nada*?

Cambiar un umbral no obliga a repetir nada. El pipeline guarda los números, no los juicios: al volver a lanzarlo, las seis primeras etapas se saltan y solo se vuelve a dictar el veredicto. Así se ve:

```
sppultra    hecho, se salta
design       hecho, se salta
build       hecho, se salta
ran         hecho, se salta
collected  hecho, se salta
wfc         hecho, se salta
verdict     python3 -m pipeline.stages.verdict ...
| PROGRESS 100 reject, falla wfc.rho
```

Puedes dictar el veredicto suelto, sin pasar por el pipeline:

```
python3 -m pipeline.stages.verdict \
  --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39 \
  --brief ~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21/design_brief_Strategy_17-9-39.json
```

Borrar las variantes, con red

Las 5.000 variantes de una estrategia pesan gigas y no sirven para nada una vez medidas. El pipeline las borra si se lo pides con `--sweep`, o después, a mano:

```
python3 -m pipeline.cleanup --project XAUUSD --strategy "Strategy 17.9.39"
```

```
liberaría      1.204.531.200 B  pipeline/XAUUSD/Strategy_17-9-39/variants
1.204.531.200 B en 1 rutas  (nada borrado: falta --apply)
```

Sin `--apply` no borra nada, solo te dice cuánto liberaría. Y antes de borrar comprueba tres cosas, y si falla cualquiera se niega:

1. Que la etapa de recogida haya **terminado**.
2. Que cada fichero exportado **siga siendo exactamente el que se exportó** — lo vuelve a hashear. Si no coincide: `Strategy 0.0.1: el export ya no coincide con su hash:`
`.../metrics_placeholder.json`
3. Que lo que va a borrar esté **dentro de la carpeta de esa estrategia**. Si el registro propone otra cosa, se para: `Strategy 0.0.1: el registro propone borrar fuera de su carpeta:`
`.../raw/XAUUSD/SPP_IS`

⚠ **La estrategia madre no se borra nunca**. No es una promesa: la madre vive en el databank de SQX y en `raw/`, y el barrido no puede salir de `pipeline/<proyecto>/<estrategia>/`. No hay ruta por la que pueda llegar hasta ella.

Lo borrado queda apuntado en el registro, con su tamaño y la hora. El registro sobrevive; los datos no. Y `python3 -m perf.disk.report` lee esos registros para decirte qué más se puede soltar.

Un ejemplo completo

```
# 1. Mira cómo está el disco antes de empezar
python3 -m perf.disk.report

# 2. Lanza las cinco madres del databank, borrando según termina cada una
python3 -m pipeline.run --project XAUUSD --databank SPP_IS --sweep

# 3. Al día siguiente, mira los veredictos
grep -h '"verdict"' ~/Desktop/AlgoData/pipeline/XAUUSD/*/verdict.json
```

Si se corta a mitad, se relanza el mismo comando y sigue donde estaba.

Qué NO te dice

- **No te dice que la estrategia sea buena**. Dice que pasó unos umbrales que has puesto tú. Los números de los que salen esos umbrales están en `verdict.json`, y son lo que hay que mirar.
- **Hoy no te dice casi nada**, porque cinco de las siete etapas son marcadores de posición con cifras inventadas. Un `proceed` de hoy no significa nada; lo que hoy está probado es el encadenado.
- `costs_provisional: true` **invalida cualquier cifra de dinero**. Mientras esté ahí, los costes son los de por defecto de SQX.
- **No vigila SQX**. Si SQX se queda colgado durante una etapa, el pipeline se queda esperando; el registro seguirá diciendo el último porcentaje que le llegó.

Si algo falla

AlgoData 7.2 GB de 60 GB. snapshots 4.8/2 GB. Libera espacio o sube el presupuesto El pipeline se ha negado a empezar porque una rama de `AlgoData` está por encima de su techo. Mira qué se puede soltar con `python3 -m perf.disk.report`, o sube el número en `perf/config.yaml`. Es la puerta que existe para que una corrida de días no acabe con el disco lleno.

sppultra: falta la entrada raw/XAUUSD/SPP_IS No está el export. Córrelo (ver *Antes de empezar*) y relanza.

design: terminó sin escribir pipeline/XAUUSD/.../design.json La etapa acabó sin error pero no dejó lo que tenía que dejar. El pipeline no la da por buena: un código de salida cero no es prueba de nada.

build: progress went 80 -> 40, it is monotonic Un fallo de la etapa, no tuyo. Apúntalo y dilo.

sppultra: design_brief_...json ha cambiado desde que se registró (brief_hash d081df20... -> e6d1be4a...) El diseño de las variantes ha cambiado **después** de haberlas fabricado. Todo lo que viene detrás estaría midiendo un diseño contra los números de otro, y eso no da error: da resultados que parecen buenos. Por eso se para. O borras la carpeta de esa estrategia en `~/Desktop/AlgoData/pipeline/` para rehacerla entera, o recuperas el brief anterior.

ran: canaries_failed = 3. un canario fallido significa que las variantes que corrió SQX no son las que se escribieron... No es que la estrategia sea mala: es que el experimento no vale. Las variantes que volvieron de SQX no son las que se le dieron — renombrado en colisión, un databank que se sincronizó por medio, algo así. Se para ahí aposta, para no gastar horas midiendo algo que no significa nada.

assets/symbols/XAUUSD.yaml: spread, commission sin valor acordado El activo no tiene costes decididos y no se arranca sin ellos. Corre `python3 -m core.assets XAUUSD` para ver qué falta. Ojo: un valor **provisional** no bloquea — es una decisión tuya, y el registro la sella con `costs_provisional: true`.

Comprobar que el pipeline sigue siendo un pipeline

```
python3 -m pipeline.verify.selftest
```

Monta una estrategia de mentira, la corre entera, y comprueba dos cosas: que el progreso se escribe **mientras** la etapa corre y nunca baja, y que si matas el proceso a mitad el registro no se corrompe y al relanzar continúa donde estaba. Tarda unos 15 segundos y termina en `0 problemas`.

80. Autopiloto — el workflow de un proyecto de punta a punta, sin pararse

Qué pregunta responde

¿Puede correr el workflow entero solo, sin que nadie (ni tú ni Claude) tenga que mirar entre paso y paso? El autopiloto lanza cada tarea de SQX y cada análisis de Python en el orden de `WORKFLOW.md`, y en cada paso que juzga (8, 10, 12, 14, 16) **decide él**: lee todos los números que han escrito los estudios, aplica tus criterios, borra en SQX lo que no pasa y sigue.

Cada estrategia sale de un paso con una de tres notas: **Pasa, Limbo o No pasa**. Las del limbo siguen, marcadas. Un paso sin criterios deja pasar a todas.

Cuándo lo usas, y cuándo no

Úsalo cuando quieras que el proyecto avance del paso 7 al 16 sin intervenir. Hoy está en **modo desarrollo**: en el paso 8 se queda con **5 estrategias al azar** y no filtra nada más. Sirve para comprobar que la cadena funciona, y rápido. **No te dice qué estrategias son buenas**.

No lo uses en un proyecto Trade_ con el modo desarrollo encendido sin saber lo que hace: el sorteo borra del databank OOS todas menos cinco.

No hace los pasos 1-5 (idea, plantilla, proyecto), ni el 16.5 en adelante: se para antes del 16.5.

Antes del autopiloto: los tres expertos

Los pasos 1 a 6 son los que más pesan: la idea, la plantilla y los building blocks. Los hace un experto en cada uno, sin escatimar tokens (dueño, 2026-10-01):

paso	experto	qué entrega
1 · idea	ideaExpert	AlgoData/ideas/<activo>/<fecha>-<nombre>.md : tres ideas exactas, con su mecanismo, su coste y su evidencia solo en el tramo de construcción ; tú eliges una
2-3 · bloques y plantilla	templateArchitect	la plantilla en AlgoData/templates/library/<nombre>/, comprobada bloque a bloque contra la idea
6 · building blocks	buildingBlocksExpert	una paleta en sqx/blocks/palettes/<nombre>.yaml : qué condiciones puede combinar el builder con la idea, y por qué

La paleta se escribe en la tarea Build del proyecto con el worker parado:

```
python3 -m sqx.projects.buildingblocks --project Test_XAUUSD_ejemplo_H1 --palette ruptura_base --role custodian
```

Responde cuántos bloques quedan encendidos y si las condiciones están dentro de tu banda (90-170). Sin este paso, el hueco aleatorio sortea entre las ~436 condiciones del donante.

Antes de empezar

- El proyecto existe, creado con `sqx.projects.builder --workflow` , en un worker (nunca en el maestro).
- El worker está **parado y libre**: el autopiloto se niega si está encendido o lo tiene otra sesión.
- Los criterios están en `pipeline/autopilot/criteria.yaml` . Hoy están vacíos y `dev.on: true` .

Cómo se ejecuta

Mira primero qué haría, sin correr nada:

```
python3 -m pipeline.autopilot.run --project Test_USDJPY_donchianUpperCrossUp_H1 --plan
```

Y luego en serio:

```
python3 -m pipeline.autopilot.run --project Test_USDJPY_donchianUpperCrossUp_H1
```

flag	obligatorio	qué hace
--project	sí	el proyecto, tal como sale en <code>registry.csv</code>
--plan	no	solo enseña los pasos que correría y dónde se pararía

Toca SQX: arranca y para el worker del proyecto en cada paso de SQX. Tarda lo que tarden las tareas; de lo que añade entre paso y paso aún no hay medida (OPEN.md #90).

Desde Claude: `/autopilot` . Lo lanza en segundo plano y solo lee el resumen, nunca el log de SQX. Por eso gasta pocos tokens.

Qué produce

Una carpeta por cada vez que se corre, `~/Desktop/AlgoData/autopilot/<proyecto>/<fecha-hora>/` :

archivo	qué es
<code>estado.txt</code>	una línea: en qué paso va ahora mismo
<code>resumen.md</code>	una fila por acción: paso, tipo, segundos y qué hizo; al final, dónde se paró
<code>fallo.md</code>	solo si algo falla: el paso, el motivo y las últimas líneas del error
<code>plan.json</code>	lo que planeó correr
<code>paso-<n>/hechos.parquet</code>	todos los números de los estudios de ese paso, una fila por estrategia y número
<code>paso-<n>/verdict.csv</code>	la nota de cada estrategia (Pasa / Limbo / No pasa), si se quedó y por qué

Borra estrategias del databank que lee la siguiente tarea de SQX (en el paso 8, `00S`). Antes de borrar, `curate` deja la lista de lo que había en `reports/<proyecto>/00S/<día>/curate/` .

Cómo se lee el resultado

`resumen.md` dice, paso a paso, cuántas estrategias entraron y cuántas salieron. Por ejemplo, la línea del juicio del paso 8 será del estilo `00S: 60 → 5 por sorteo de 5 (semilla ...)`. La semilla queda escrita, así que el sorteo se puede repetir.

Para escribir un criterio necesitas saber qué números hay. Esta orden los lista, con cuántas estrategias lo tienen, su mediana, su mínimo y su máximo:

```
python3 -m pipeline.autopilot.facts --project Test_USDJPY_donchianUpperCrossUp_H1 --step 8 --grep gate.
```

Salida real (2026-10-01):

	count	median	min	max
key				
gate.scorecard.degradacion_value	276	0.7280	0.0070	2.3325
gate.scorecard.estaticas_value	296	15766.8999	-16655.2598	41052.2305
gate.scorecard.familia_value	189	0.1734	0.0170	0.4203
gate.scorecard.forma_value	190	1.7351	0.6008	2.9440
gate.scorecard.mono_p	190	0.1747	0.0170	0.6407
gate.scorecard.sanidad_value	300	327.0000	250.0000	660.0000
gate.scorecard.survives	300	1.0000	0.0000	1.0000

Un criterio en `criteria.yaml` se escribe así:

```
"8":
  cut: 00S
  rules:
    - {fact: gate.scorecard.mono_p, pass: "<= 0.05", limbo: "<= 0.10"}
    - {fact: gate.scorecard.forma_value, near_median: 0.2, limbo_near_median: 0.5}
```

La primera regla es un umbral. La segunda dice: pasa si está a menos del 20 % de la mediana, y queda en limbo hasta el 50 %. La mediana se calcula sobre las estrategias del databank que se va a cortar. La nota de una estrategia es **la peor de sus reglas**. Si a una estrategia le falta el número de una regla, queda en limbo en esa regla.

El paso 12 (cross timeframe) juzga madres, no hermanas. Cada madre tiene sus números por timeframe y por papel: `crossTF.cells.scaled.H4.sharpe` es el Sharpe de su versión escalada a H4, `crossTF.cells.unscaled.D1.pf` el PF sin escalar en D1, `crossTF.cells.control.H4.p` el control de la escalada a H4, y `..._vs_original` ese número dividido por el de la madre en su timeframe. También `crossTF.reading.n_survives`: en cuántos timeframes sobrevivió. Si la madre pasa, **sigue solo ella**, en su timeframe y sin escalar; las versiones escaladas se borran siempre. Por ejemplo:

```
"12": {cut: CrossTF, rules: [{fact: crossTF.reading.n_survives, pass: ">= 1"}]}
```

Sobre las 15 madres del proyecto de prueba, esa regla deja pasar 2 (salida real, 2026-10-01).

Dónde corta cada paso: siempre en el databank del que bebe lo que viene después.

paso	corta	por qué ahí
8	00S	lo lee el retest de mercados (9)
10	Retest Markets - Family	de ahí salen las madres que se escalan (10.5)
12	CrossTF	de ahí copia el 13 las madres, sin las escaladas
14	MCR 8 Stress	es el único de los ocho MCR que lee el SPP
16	SPP 00S	lo lee la WFM (19); las que quedan se casan con su gemela de SPP IS y las variantes (16.5) se diseñan con el backtest combinado

Un ejemplo completo

El plan sobre un proyecto donde todo, del 7 en adelante, está por hacer (salida real, simulando ese estado sobre `Test_USDJPY_donchianUpperCrossUp_H1`):

```
7 sqx · 8 python · 8 judge · 9 sqx · 10 python · 10 judge · 11 sqx · 12 python · 12 judge ·
13 sqx · 14 python · 14 judge · 15 sqx · 16 python · 16 judge → se para antes del 16.5
```

Sobre ese mismo proyecto, que ya está terminado, el plan real no corre nada:

```
{"do": [], "stop": {"n": "25.5", "why": "no tiene tareas de SQX que el autopiloto sepa lanzar"}}
```

⚠ Todavía no se ha corrido nunca de verdad contra SQX (OPEN.md #90). Cuando se corra, aquí irá su `resumen.md`.

Qué NO te dice

- En modo desarrollo, **nada sobre la calidad de las estrategias**: las cinco que pasan son azar.
- El veredicto propio de cada estudio (el MANTENER/DESCARTAR de la puerta, por ejemplo) **no corta por sí solo**. Es un número más (`gate.verdict_csv.drop`) y solo corta si lo pones en una regla.
- Hoy, en el paso 12, **no puede comparar con la original**: en el último run la celda de la madre en su propio timeframe salió con 0 operaciones (OPEN.md #89). Los números `..._vs_original` faltan hasta que eso se arregle.

Si algo falla

- «**el worker no está libre**»: está encendido o lo tiene otra sesión. Espera, no lo pares tú.
- «**paso N: no se lanza**»: el preflight del paso de SQX dice qué falta (una tarea, una entrada vacía).
- «**tiene reglas y criteria.yaml no dice qué databank cortar**»: falta `cut` en ese paso.

En todos los casos `fallo.md` dice el paso y el motivo, y el worker queda parado.

55. Retirar proyectos — quitar de SQX los custom projects que ya respondieron

Qué pregunta responde

¿Qué custom projects hay en cada instalación, cuántas tareas tiene cada uno, cuánto ocupa y para qué se creó? Y, cuando uno ya no sirve, lo quita de SQX sin perder su configuración: el `project.cfx` se guarda comprimido en AlgoData y la carpeta desaparece de `user/projects`.

Lo normal: no haces nada

Cada lunes a las 03:00 (la noche del domingo al lunes) el agente `projectJanitor` pasa solo (`bin/weekly-project-cleanup.sh`, en el crontab). Aplica la regla de `sqx/projects/sweep.py`:

se retira	se queda
todo proyecto <code>Test_</code> de w1 y w2	todo proyecto <code>Trade_</code>
todo proyecto antiguo de menos de 10 tareas (un workflow entero tiene 15-18)	lo tocado en las últimas 24 h
lo que tú apuntes en <code>AlgoData/projects/retire-queue.txt</code>	todo lo de una instalación encendida
	del máster, todo lo que no esté en la cola

Antes de retirar, el agente busca señales de vida que la regla no ve: un encargo abierto que nombre el proyecto, una línea de `OPEN.md`, o un ledger o un pipeline escritos esa semana. Si encuentra alguna, se queda con la duda y no lo toca. No arranca, no para ni consulta ningún SQX.

Te deja el resumen en `AlgoData/audit/AAAA-MM-DD-proyectos.md`: qué se fue, qué se quedó y por qué, y cuánto disco se liberó. El log está en `AlgoData/logs/weekly-project-cleanup.log`.

Para que se lleve un proyecto concreto, incluido uno del máster, añade una línea a la cola:

```
custodian USDJPY_emaCross_H1
master    XAU_crossmarket_demo
```

Una vez retirado, su línea desaparece de la cola.

Cuándo lo usas a mano, y cuándo no

- **Al terminar una tarea que creó un proyecto `Test_`**. Quien lo crea, lo retira. Es la regla.
- **Cuando `--list` enseña proyectos de pocas tareas** que quedaron de pruebas sueltas.
- **No** para un proyecto `Trade_`: ese se queda hasta que tú digas.
- **No** en el máster, salvo un proyecto que tú hayas nombrado.
- **No** para "hacer sitio" en un proyecto que se va a volver a correr: vacía sus databanks, no lo borres.

Lo que un run encontró **no vive en el proyecto**. Métricas, trades, curvas y veredictos ya están en parquet en `AlgoData/raw/`, `harvest/` y `reports/`, y retirar el proyecto no los toca. Lo único que se pierde son las estrategias de los databanks. Si quieres conservar las de la última prueba (la WFM), añade `--keep WFM`.

Antes de empezar

- **La instalación del proyecto tiene que estar parada.** Un SQX encendido tiene el proyecto en memoria y lo reescribiría al salir (regla dura 4). El comando lo comprueba y se niega si no lo está: el puerto en los workers, cualquier proceso que corra desde `~/Desktop/SQX` en el máster.
- **Nadie puede estar usándolo.** Si el proyecto cambió hace menos de 60 minutos, el comando se niega, porque puede haber otra sesión cosechándolo.

Cómo se ejecuta

Ver qué haría el pase semanal si corriera ahora (no toca nada):

```
python3 -m sqx.projects.retire --sweep
```

Ver qué hay:

```
python3 -m sqx.projects.retire --list
```

Retirar (sin `--yes` solo dice lo que haría):

```
python3 -m sqx.projects.retire Test_XAU_keltner_humo --role custodian          # prueba en seco
python3 -m sqx.projects.retire Test_XAU_keltner_humo --role custodian --yes    # de verdad
python3 -m sqx.projects.retire Trade_XAU_keltner_H1 --role custodian --keep WFM --yes
```

flag	obligatorio	qué hace
nombres	para retirar	uno o varios proyectos de la misma instalación
<code>--list</code>	no	lista los custom projects de las tres instalaciones y no toca nada; es lo que hace sin nombres
<code>--role</code>	no	<code>custodian</code> por defecto; <code>conductor</code> o <code>master</code>
<code>--keep</code>	no	un databank que se guarda junto al <code>.cfx</code> , p. ej. <code>--keep WFM</code> ; se puede repetir
<code>--sweep</code>	no	aplica la regla semanal a las tres instalaciones; con <code>--yes</code> , la ejecuta
<code>--yes</code>	no	lo hace de verdad. Sin él, prueba en seco

Tarda segundos y no arranca SQX.

Qué produce

- `~/Desktop/AlgoData/projects/retired/<instalación>/<proyecto>-<fecha>.tar.gz`: el `project.cfx` y los databanks de `--keep`.
- La carpeta `<instalación>/user/projects/<proyecto>` **se borra**.
- En `~/Desktop/AlgoData/projects/registry.csv` la fila del proyecto queda con la fecha de retiro y la ruta del archivo. Si el proyecto era anterior al registro, se le añade una fila.

El registro y los nombres

Desde el 2026-09-26, `sqx.projects.builder` solo acepta dos prefijos:

- **Test_...** : una prueba de funcionamiento. Se retira en cuanto ha respondido.
- **Trade_...** : un proyecto con intención de dar frutos reales. Se queda.

Además pide `--purpose "una frase"` . Cada proyecto que crea deja una fila en `registry.csv` : nombre, tipo, instalación, fecha, propósito, símbolo, timeframe, plantilla y si lleva el workflow entero. En `--list` , la columna `legacy` marca los proyectos anteriores a esta regla.

Cómo se lee el resultado

```
$ python3 -m sqx.projects.retire --list
== master (SQX) - 13 custom
AUDJPY                19 tareas          221 MB  2026-09-25 07:07  legacy
CADJPY_H1             19 tareas           2 MB  2026-09-02 14:33  legacy
EURJPY_H1             18 tareas          23 MB  2026-09-25 07:07  legacy
EURUSD                19 tareas         584 MB  2026-09-25 07:04  legacy
GBPJPY_H1             20 tareas        1066 MB  2026-09-25 07:07  legacy
Infinox_SP500ft_H4_HighPrecision  8 tareas          82 MB  2026-09-02 14:33  legacy
SP500_H1              18 tareas         197 MB  2026-09-25 07:00  legacy
TestXAUUSD2           3 tareas           0 MB  2026-09-23 20:32  legacy
USDCHF                23 tareas          10 MB  2026-09-25 06:59  legacy
USDJPY                19 tareas        1433 MB  2026-09-25 07:02  legacy
XAUUSD                17 tareas         734 MB  2026-09-25 07:10  legacy
XAUUSD_Breakout_H1     5 tareas           0 MB  2026-09-03 06:50  legacy
XAU_crossmarket_demo   3 tareas           0 MB  2026-09-23 20:34  legacy
== conductor (SQX_w1) - 2 custom
XAUUSD_Breakout_H1     5 tareas           0 MB  2026-09-02 21:12  legacy
XAU_crossmarket_demo   3 tareas           0 MB  2026-09-23 20:48  legacy
== custodian (SQX_w2) - 17 custom
PerfUSDJPY_Python_v1   15 tareas        1393 MB  2026-09-24 20:58  legacy
TestUSDJPY_Workflow_v1 15 tareas        207 MB  2026-09-24 20:05  legacy
TestUSDJPY_crossTF_v1  1 tareas         26 MB  2026-09-24 12:56  legacy
TestXAUUSD2            4 tareas           2 MB  2026-09-24 06:45  legacy
TestXAU_crossTF        1 tareas           1 MB  2026-09-23 21:23  legacy
Test_XAUUSD            1 tareas           0 MB  2026-09-22 13:55  test
USDJPY_emaCross_H1     18 tareas        386 MB  2026-09-26 07:28  legacy
USDJPY_structural_v1   18 tareas          9 MB  2026-09-26 08:09  legacy
USDJPY_variantes       3 tareas           0 MB  2026-09-25 16:42  legacy
USDJPY_wfm             1 tareas          34 MB  2026-09-25 17:50  legacy
USDJPY_workflow_profiling_v1 18 tareas       2496 MB  2026-09-26 13:38  legacy
XAU_ISO0S_ejemplo      2 tareas          26 MB  2026-09-23 14:54  legacy
XAU_crosstf_probe      2 tareas           1 MB  2026-09-23 20:55  legacy
XAU_mcr_prueba         8 tareas          33 MB  2026-09-23 23:23  legacy
algo_XAU_keltner       15 tareas          1 MB  2026-09-23 10:10  legacy
blocktest_grupoA       1 tareas           6 MB  2026-09-24 08:49  legacy
smoke_keltnerUpperCrossUp 1 tareas          6 MB  2026-09-23 07:26  legacy
```

Cada fila es un proyecto: nombre, tareas, tamaño en disco, último cambio, tipo y propósito (si está registrado). Con el workflow entero, un proyecto tiene 15 a 18 tareas. **Uno de 1 a 5 tareas es una prueba suelta**, y es lo primero que conviene retirar.

```
$ python3 -m sqx.projects.retire --role custodian smoke_keltnerUpperCrossUp XAU_mcr_prueba
would retire SQX_w2/smoke_keltnerUpperCrossUp: 1 tareas, 6 MB -> /home/sergioguslw/Desktop/AlgoData/projects/retired/SQX_w2/smoke_keltnerUpperCrossUp-2026-09-26.tar.gz
would retire SQX_w2/XAU_mcr_prueba: 8 tareas, 33 MB -> /home/sergioguslw/Desktop/AlgoData/projects/retired/SQX_w2/XAU_mcr_prueba-2026-09-26.tar.gz
dry run - add --yes to retire them
```

La prueba en seco dice, por proyecto, cuántas tareas y megas se van y a qué archivo irían.

Un ejemplo completo

1. `python3 -m sqx.projects.retire --list:ves smoke_keltnerUpperCrossUp` en el custodio, con 1 tarea y 6 MB.
2. Compruebas que el custodio está parado (`bin/sqx-worker.sh --role custodian status`) y que ninguna otra sesión lo está usando.
3. `python3 -m sqx.projects.retire smoke_keltnerUpperCrossUp --role custodian`: la prueba en seco.
4. Lo mismo con `--yes`. La carpeta desaparece y su `.cfx` queda en `AlgoData/projects/retired/SQX_w2/smoke_keltnerUpperCrossUp-<fecha>.tar.gz`.

Para recuperarlo: `tar xzf <archivo> -C ~/Desktop/SQX_w2/user/projects/`, con el custodio parado. Vuelve con su configuración y sin estrategias (salvo las de `--keep`).

Qué NO te dice

- **Si un proyecto sigue haciendo falta.** El número de tareas y la fecha orientan, pero no saben si una ficha de knowhow cita ese proyecto como ejemplo. La ficha sigue siendo válida porque los datos están en parquet, aunque el proyecto ya no esté en SQX.
- **Si otra sesión lo va a usar mañana.** La guarda de 60 minutos solo protege lo que se está moviendo ahora.

Olvidar un proyecto del todo: `sqx.projects.forget`

`retire.py` deja adrede el parquet y los CSV de `raw/`, `harvest/`, `metrics/`, `reports/`, `structural/`, `strategyPermutations/` y `atrCalculator/` — es lo que un run encontró, y normalmente vale la pena conservarlo. Cuando de verdad quieres borrar un proyecto entero, no solo sacarlo de SQX, usa `sqx.projects.forget`: retira lo que siga vivo en el conductor o el custodio (igual que `retire.py`, así que el `.cfx` queda archivado) **y además** borra su subcarpeta en esas seis carpetas de AlgoData.

```
python3 -m sqx.projects.forget USDJPY_wfm          # prueba en seco: qué se borraría
python3 -m sqx.projects.forget USDJPY_wfm --yes    # de verdad
```

Se niega igual que `retire.py` con un proyecto de serie, y con uno que solo exista en el máster — ese install es del dueño (regla dura 3) y nada lo toca sin que él lo nombre. No borra `bars/`, `spread/`, `templates/`, `ledger/`, `projectsBackup/`, `snapshots/`, `logs/`, `profiling/`, `cache/` ni `pipeline/`: eso es dato base o el registro global, nunca de un proyecto suelto.

Si algo falla

- `the custodian is up ...`: el worker está encendido. Páralo con `bin/sqx-worker.sh --role custodian stop`, pero solo si no hay nadie más usándolo.
- `... changed less than 60 min ago`: alguien lo está tocando. Mira `ListAgents` y el log del día.
- `... is a stock project`: `Builder`, `Retester` y los demás de serie no se retiran nunca.

62. Metadatos de una estrategia — qué hace y con qué costes se probó, leído del fichero

Qué pregunta responde

Abre un `.sqs` y te dice, como campos sueltos y sin arrancar SQX: si opera en largo, en corto o en los dos sentidos; qué condiciones tiene la entrada (y la salida, si la hay) con sus valores reales; qué tipo de orden usa y si lleva stop, objetivo, trailing o salida por número de barras; con qué money management se calcularon sus números; si cierra el viernes y a qué hora; y con qué spread, slippage, comisión y swap se hizo el último test. Es lo que pinta el panel de metadatos de la ficha de estrategia de la ventana (encargo 22 \$5 y \$12.2) y lo que el archivo de estrategias congela.

Cuándo lo usas, y cuándo no

Cuando quieres saber qué es una estrategia antes de gastar CPU en ella, o comprobar que el coste con el que se hizo un backtest es el que crees.

No es un traductor: no te da pseudocódigo ni un backtest en Python (eso quedó fuera de esta ronda, decisión del dueño del 2026-09-27). Da **campos**. Tampoco te dice si la estrategia es buena: no mira ni una métrica.

Antes de empezar

Nada. Solo lee ficheros del disco: el `.sqs` y, si se lo das, el `project.cfx` del proyecto. **No envía ningún comando a SQX**, así que puedes usarlo con el maestro abierto, con un worker corriendo o con todo apagado. Da igual de qué instalación sea el fichero.

Cómo se ejecuta

```
python3 -m sqx.inspect.strategymeta
"$HOME/Desktop/SQX/user/projects/XAUUSD/databanks/00S/Strategy 10.13.25.sqs" \
  --cfx "$HOME/Desktop/SQX/user/projects/XAUUSD/project.cfx"
```

argumento	obligatorio	qué hace
el <code>.sqs</code>	sí	la estrategia. Lo normal es que esté dentro de la carpeta de un databank
<code>--cfx</code>	no	el <code>project.cfx</code> del proyecto al que pertenece ese databank. Con él, además de lo que guarda el <code>.sqs</code> , sale lo que dice hoy la tarea que escribe en ese databank

Tarda menos de un segundo por estrategia. Imprime un JSON por pantalla y no escribe nada. El código vive en `sqx/inspect/strategymeta/` (`__init__.py`, `rules.py`, `settings.py` y `sqx/inspect/strategymeta/__main__.py`, que es la línea de comandos). Desde Python, lo mismo es `from sqx.inspect.strategymeta import read` y `read(Path(sqs), Path(cfx))`.

Qué produce

Solo la salida por pantalla: un diccionario con estos campos.

campo	qué es
name , feed , timeframe	nombre, feed y timeframe del último test
direction	long , short o both : la dirección de las órdenes de entrada
entries , exits	una fila por condición: la señal, el operador (AND / OR), el bloque y sus parámetros con el valor real (no el nombre de la variable)
signals	cada señal en una línea: Bloque(Param=valor, ...) AND Bloque(...)
indicators	el resumen que el propio SQX guarda de indicadores de entrada y de salida
orders	cada orden de entrada: tipo, dirección, stop, objetivo, trailing, break-even y salida por barras (null = no lleva)
order_types	los tipos de orden: EnterAtMarket , EnterAtStop , EnterAtLimit
money_management	el método de tamaño con sus parámetros y el capital inicial; from dice si es el de la estrategia o el de la tarea
friday_close	si cierra el viernes (on), a qué hora y la clave de SQX (ExitOnFriday)
trading_options	fin de semana, cierre al final del día, franja horaria, máximo de operaciones al día, sesión
last_test	la configuración con la que SQX calculó los números que guarda el .sqx : databanks de entrada y salida, ventana, tramo OOS, precisión, motor y costes (spread, slippage, distancia mínima, comisión, swap)
backtest	solo con -- cfx : cada tarea del proyecto que escribe en ese databank, con la misma forma que last_test y si está activa
asset , asset_card_sha256	la ficha de assets/symbols/ del activo y su huella de hoy
notes	avisos que hay que leer junto a los números

Cómo se lee el resultado

La primera mitad dice qué hace la estrategia:

```
$ python3 -m sqx.inspect.strategymeta ".../XAUUSD/databanks/005/Strategy 10.13.25.sqx" --cfx .../XAUUSD/project.cfx
{
  "name": "Strategy 10.13.25",
  "feed": "XAUUSD_DukasM1_Infinox",
  "timeframe": "M30",
  "direction": "long",
  "signals": {
    "LongEntrySignal": "CBlock_WassersteinShiftLow(Int2=50, Double3=16.8) AND BBBarOpensAboveUp(Period=20, Deviation=1.9, ComputedFrom=0, Shift=1)"
  },
  "indicators": {
    "entry": [
      "(BB) Bar opens above Upper band",
      "WassersteinShiftLow"
    ],
    "exit": [
      "ExitAfterXBars"
    ]
  },
  "orders": [
    {
      "type": "EnterAtMarket",
      "direction": "long",
      "stop_loss": null,
      "profit_target": null,
      "trailing_stop": null,
      "break_even": null,
      "exit_afterBars": 22
    }
  ],
  "order_types": [
    "EnterAtMarket"
  ],
  "friday_close": {
    "on": true,
    "key": "ExitOnFriday",
    "time": "21:00"
  }
}
```

La Strategy 10.13.25 del proyecto XAUUSD compra (long) a mercado cuando se cumplen a la vez dos condiciones: el bloque propio WassersteinShiftLow con 50 y 16,8, y que la barra abra por encima de la banda de Bollinger superior (periodo 20, desviación 1,9). No lleva stop ni objetivo: sale a las 22 barras. Cierra los viernes a las 21:00.

La segunda mitad dice con qué se probó (la captura acorta backtest , cuyos costes son iguales a los de last_test):

```
$ python3 -m sqx.inspect.strategymeta ".../XAUUSD/databanks/00S/Strategy 10.13.25.sqx" --cfx .../XAUUSD/project.cfx (segunda mitad)
{
  "money_management": {
    "from": "task",
    "method": "ATRRiskBasedSizingFixedRisk",
    "params": {
      "Amount": "1000",
      "ATRPeriod": "20",
      "ATRMult": "5",
      "Decimals": "2",
      "LotsIfNoMM": "0.01",
      "MaxLots": "5000"
    },
    "initial_capital": "100000",
    "max_drawdown": "30"
  },
  "last_test": {
    "input": "Results",
    "output": "00S",
    "window": [
      "2008.01.01",
      "2022.12.31"
    ],
    "oos": [
      [
        "2018.01.01",
        "2022.12.31"
      ]
    ],
    "costs": {
      "spread": "0",
      "slippage": "0",
      "min_distance": "10",
      "commission": [
        {
          "method": "PercentageBased",
          "value": "0.01"
        }
      ],
      "swap": {
        "use": "true",
        "type": "percent",
        "long": "-8",
        "short": "38.76",
        "tripleSwapOn": "WEDNESDAY",
        "rolloutHour": "23:00"
      }
    }
  },
  "backtest": [
    {
      "project": "XAUUSD",
      "task": "Retest strategies",
      "active": true,
      "costs": "... (iguales a last_test)"
    }
  ],
  "asset": "XAUUSD",
  "asset_card_sha256": "6aa3ff6596aceed8cf3f2947b5981e4d962fafec185775bd3cfae7b8819c4667",
  "notes": [
    "asset_card_sha256: ficha de costes actual, no la del backtest (no existe la histórica)"
  ]
}
```

Tres fuentes, tres significados, y conviene no mezclarlas:

- **last_test** es lo que de verdad se usó: SQX lo guarda dentro del `.sqx` al hacer el test. Aquí: spread 0, slippage 0, comisión del 0,01 % y swap en porcentaje.
- **backtest** es lo que la tarea del proyecto dice **hoy**. Si alguien cambió la tarea después del test, no coinciden y una nota lo avisa: «los costes de «...» hoy no son los del último test».
- **asset_card_sha256** es la huella de la ficha del activo **de hoy**. `assets/` no guarda historia, así que no es la del backtest; la nota lo dice siempre.

Si varias tareas escriben en el mismo databank (una gemela desactivada, por ejemplo), salen todas y una nota lo dice; no se elige ninguna por ti.

Un ejemplo completo

1. Eliges una estrategia de un databank, por ejemplo la carpeta `005` del proyecto XAUUSD del maestro.
2. Lanzas el comando de arriba con su `--cfx`.
3. Lees `direction` y `signals` para saber qué hace, `orders` para ver si lleva stop, y `last_test.costs` para el coste con el que se calcularon sus números.
4. Si `notes` trae algo más que el aviso de la ficha, léelo antes de fiarte de los costes.

Para una estrategia construida por el builder genérico (no desde plantilla) las señales del lado que no opera salen como `Boolean(Value=false)`: es SQX desactivando ese lado, no un error.

Qué NO te dice

- No dice si la estrategia es buena ni cuánto gana: no mira métricas.
- No traduce la lógica a pseudocódigo ni la ejecuta.
- No sabe qué ficha de costes estaba vigente el día del backtest: solo la de hoy.
- Los parámetros de un bloque propio salen con el nombre que les dio SQX (`Int2` , `Double3`), no con el significado que tienen en el bloque.
- Las horas de `trading_options.weekend` salen tal cual las guarda SQX. En el maestro aparecen como `00:38`: parece que en la tarea se escribió `2300` en un campo que va en segundos; mientras esa opción esté apagada (`on: false`) no afecta a nada.

Si algo falla

- **Una nota «la tarea «...» no se pudo leer (falta Retest-Task8.xml en el .cfx)»:** el proyecto está roto, le falta el fichero de una tarea, y no se puede saber si esa tarea escribe en el databank. `python3 -m sqx.inspect.project_health` lo confirma. El resto de la salida sigue valiendo.
- **Una tarea de backtest con costs: null** y la nota «no lleva costes en su XML»: es un Custom analysis, que no hace backtest; valen los costes de `last_test`.
- **La nota «lee sus costes de CustomData»:** es un Automatic retest (MC, SPP, WFM, retest en mercados), que guarda su propio spread y slippage aparte; `setup_from` lo dice. Si no lleva swap, `swap` sale `null`.
- **backtest: []** y la nota «ninguna tarea del proyecto escribe en este databank»: el `.sqx` no está en la carpeta de un databank de ese proyecto, o la tarea escribe en el databank por defecto. `last_test` sigue siendo válido.
- **asset: null**: ningún fichero de `assets/symbols/` declara ese feed.